Git, DevOps & CI/CD

Formation ENSTA 2A — TD1

PLATINUM Top 1%
ecovadis
Sustainability Rating
NOV 2024
ISO 27001
BUREAU VERITAS
Certification
VISA

1 Introduction au Devops
2 Git : Pourquoi et comment
3 Concepts fondamentaux Git
4 Workflow collaboratif
5 Investigation avec Git
6 Bug en production : Workflow Hotfix
7 CI/CD : Concepts
8 Pipeline CI : Composants
9 GitHub Actions
10 Introduction à Docker
11 Conclusion

LEADER DU CONSEIL EN IA, CYBER & TRANSFORMATION DIGITALE

engagé à construire une société numérique plus sécurisée et responsable

Fondé en 2000 - 100% INDÉPENDANT & 100% CONSULTANTS SALARIÉS

PLATINUM Top 1% ecovadis Sustainability Rating NOV 2024

ISO 27001 BUREAU VERITAS Certification

THE GLOBAL COMPACT

CERTIFIED CHOOSE MYCOMPANY ESG

HappyIndex® AtWork FRANCE 2024

NR LABEL NUMÉRIQUE RESPONSABLE

1200
Collaborateurs
Recrutement sur profil

+160
Clients
Grands Comptes
CAC 40, SBF 120, Public

120
M€ CA

Bruxelles
Paris

HeadMind Partners Cyber Risk & Security — **550** CONSULTANTS

Maîtriser et **sécuriser** sa transformation numérique

HeadMind Partners AI & Blockchain — **50** CONSULTANTS

Réussir la transition vers l'Intelligence Artificielle

HeadMind Partners Digital — **400** CONSULTANTS

Piloter les projets de transformation

1. Introduction au DevOps

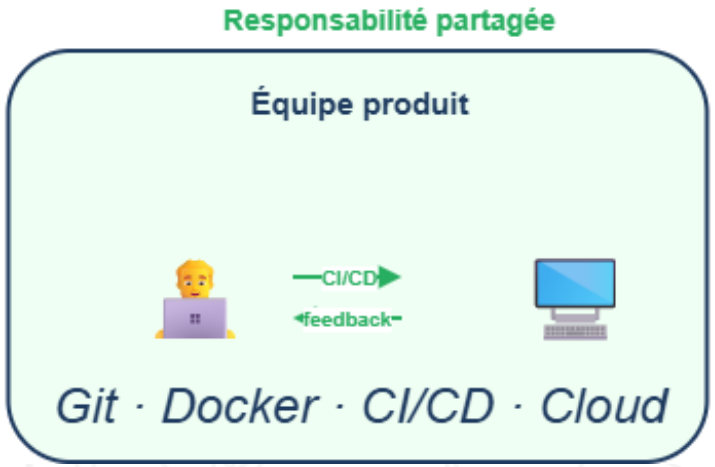
Contexte historique

Avant le DevOps :

- Ceux qui codent (développeurs) et ceux qui mettent en ligne (administrateurs) ne se parlaient pas
- Les devs veulent ajouter des fonctionnalités rapidement
- Les admins veulent que rien ne casse
- Résultat : **conflits, lenteur, bugs en production**

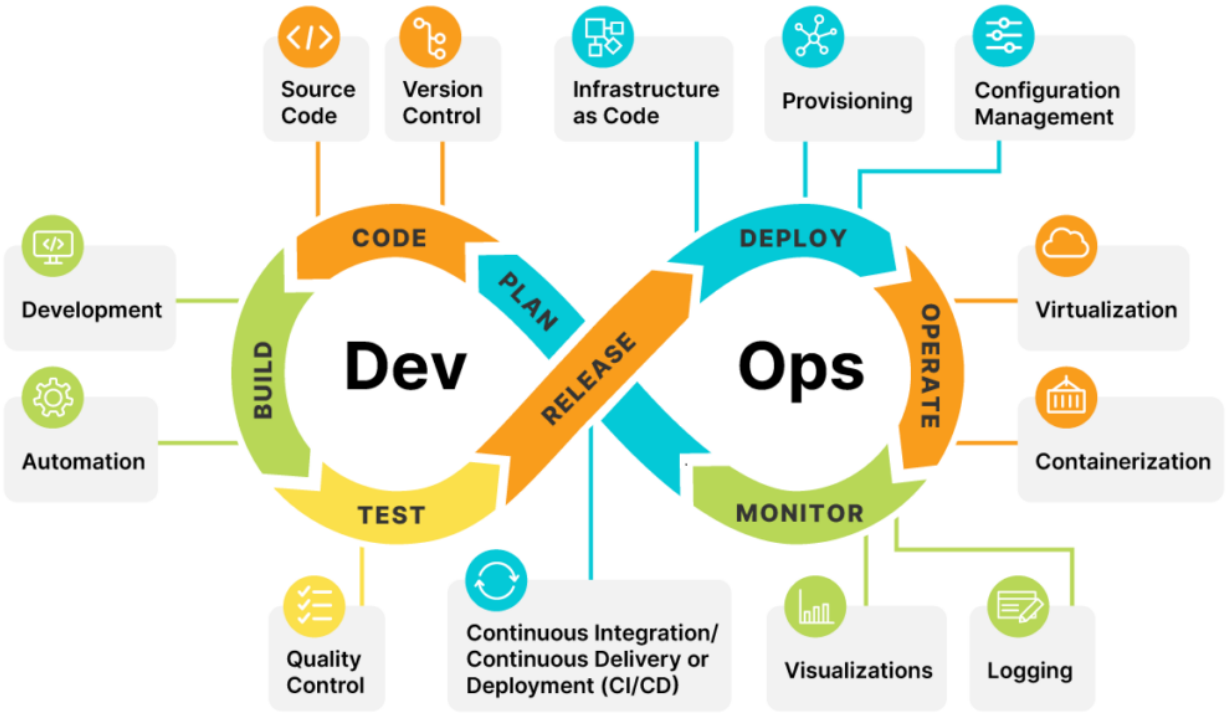
La philosophie DevOps :

- Tout le monde est responsable du produit, du code jusqu'à l'utilisateur final
- **Automatiser tout ce qui peut l'être (tests, déploiement, montée en charge, ...)**
- Feedback rapide : savoir vite si quelque chose ne marche pas
- Culture de collaboration, pas de silos



Enjeux du DevOps

- Time-to-market : livrer plus vite que la concurrence
- Scalabilité : gérer la croissance sans exploser
- Fiabilité : 99.9% uptime = 8h de downtime/an
- Coût : automatiser pour réduire les erreurs humaines
- Sécurité : DevSecOps — la sécu dès le début



Cloud : permet le DevOps

Les 3 modèles

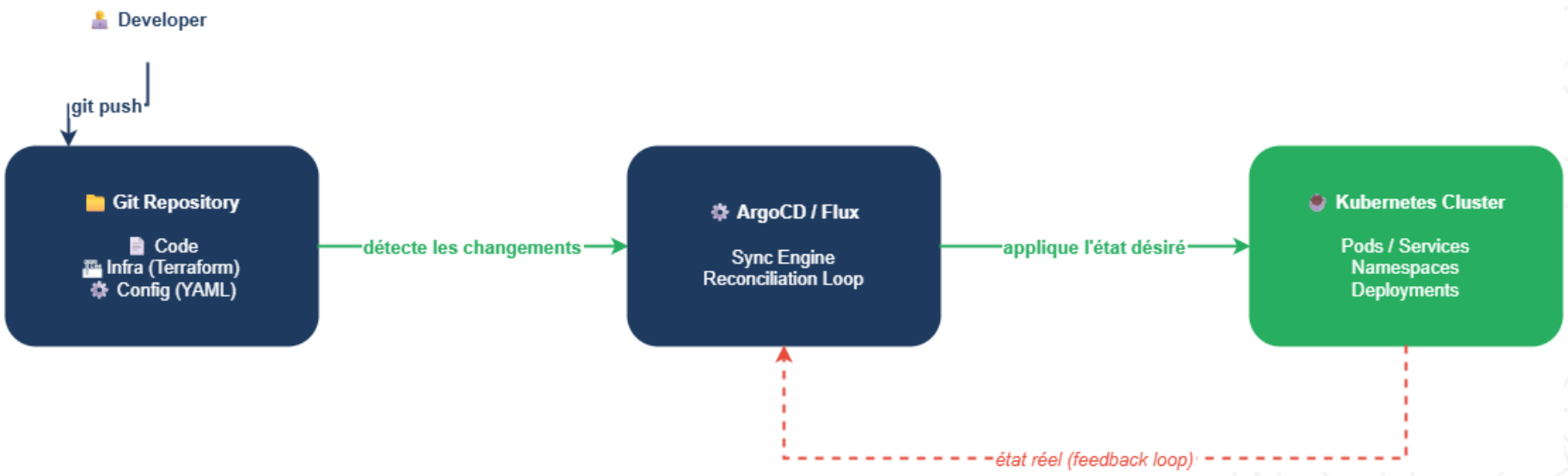
- IaaS : VMs à la demande (EC2, Azure VM)
- PaaS : Plateformes managées (Heroku, App Service)
- Serverless : Pas de serveur à gérer (Lambda, Functions)

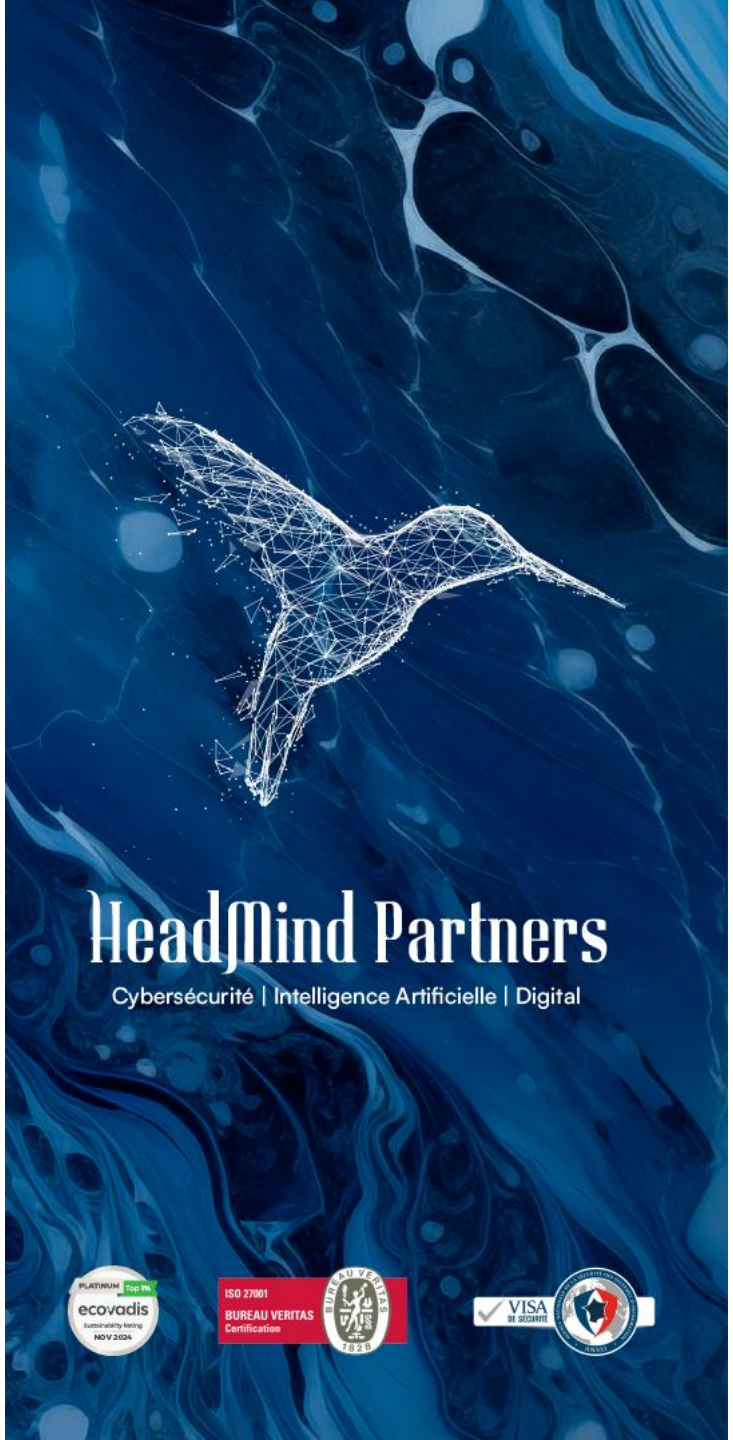
Avantages

- Pay-as-you-go : on paie ce qu'on consomme
- Élasticité : scale up/down automatique
- Rapidité : une MV en 30 secondes
- Globalisation : datacenters partout dans le monde

GitOps : Git comme source de vérité

- Principe : tout est dans Git (code + infra + config)
- Déclaratif : on décrit l'état désiré, pas les étapes
- Automatisé : push sur main → déploiement auto
- Auditable : qui a changé quoi, quand, pourquoi
- Rollback facile : git revert = retour en arrière
- Outils : ArgoCD, Flux, Terraform



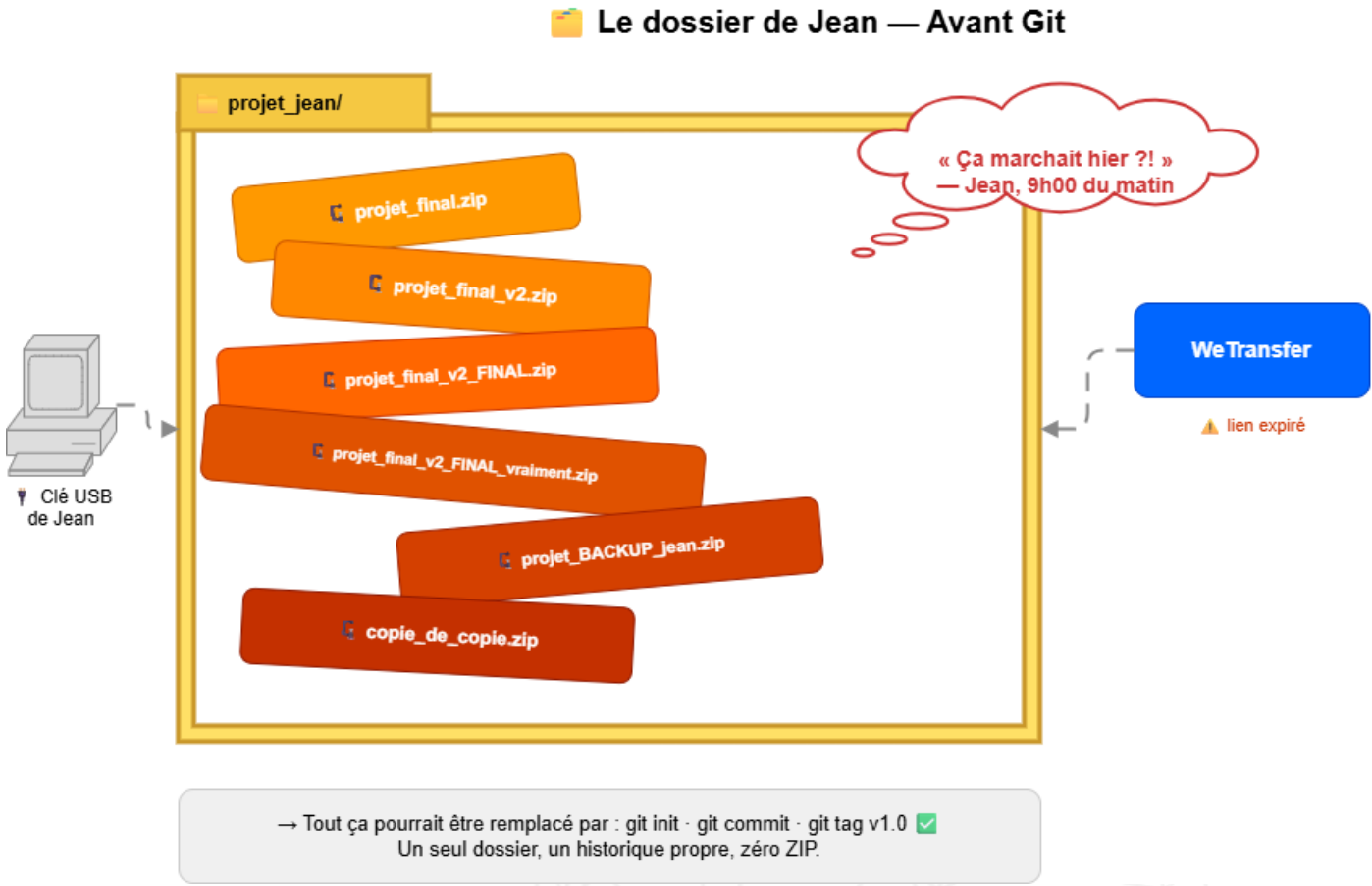


2. Git

Pourquoi et comment

Le problème sans Git

- projet_final.zip
- projet_final_v2.zip
- projet_final_v2_FINAL.zip
- projet_final_v2_FINAL_vraiment.zip
- Code échangé par clé USB ou WeTransfer
- « Ça marchait hier, qui a touché au code ? »
- Impossible de revenir en arrière proprement



Git en chiffres

2005

Créé par Linus Torvalds (en 10 jours !)

100M+

Développeurs sur GitHub

400M+

Repositories sur GitHub

Git est construit en Git

- Le repo Git (git/git) a plus de 1 million de commits
- Linus l'a créé car BitKeeper est devenu payant
- Premier commit : 4 jours après le début du projet
- Utilisé par : Microsoft, Google, Netflix, NASA...
- Le noyau Linux : 27M lignes de code, géré par Git

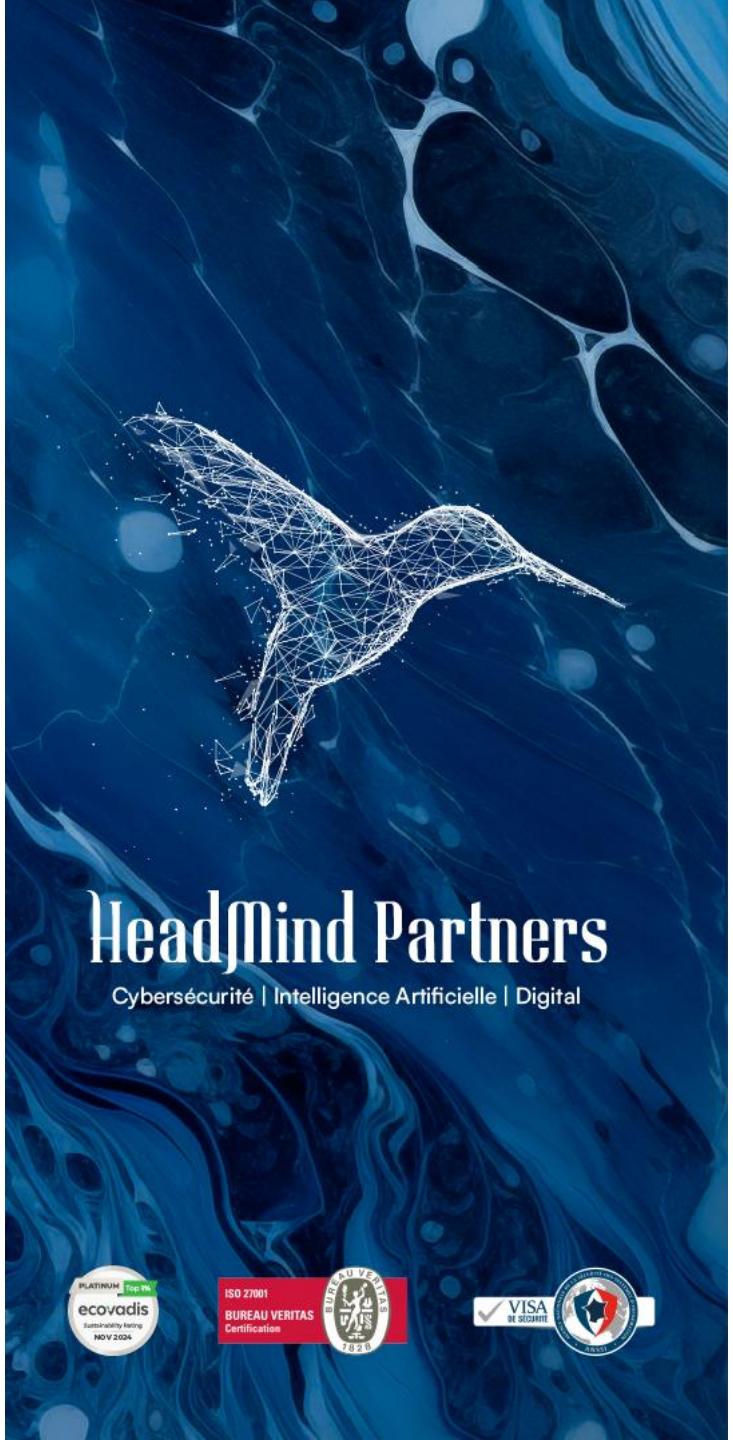
Pourquoi Git a gagné

Distribué

- Chaque clone = backup complet
- Travail offline possible
- Pas de single point of failure

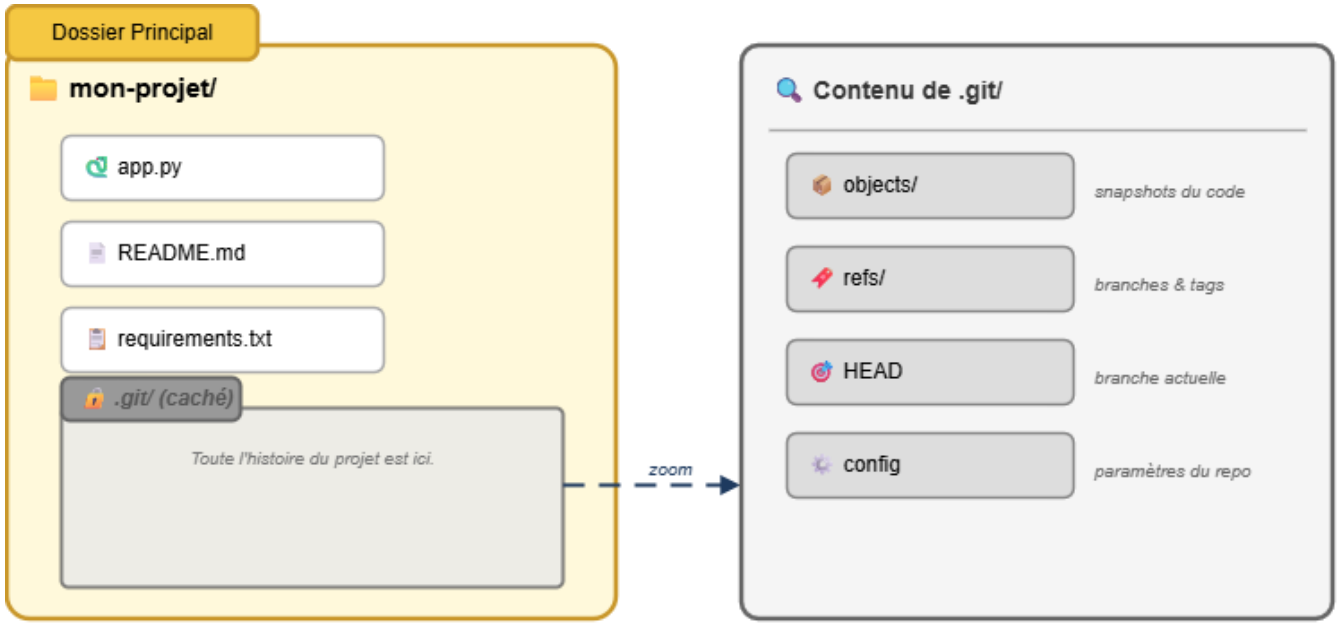
Performant

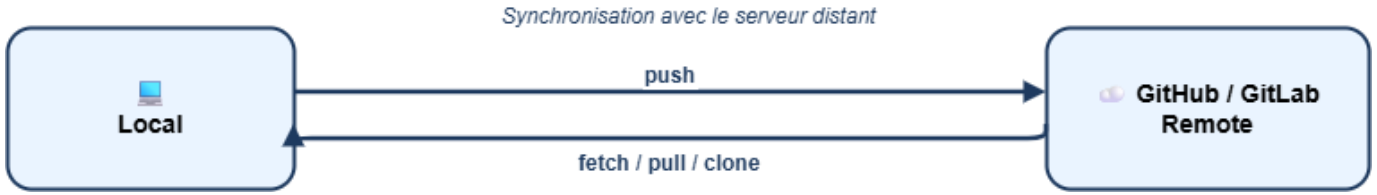
- Branching ultra rapide (pointeurs)
- Opérations locales = instantanées
- Gestion efficace des gros projets



3. Concepts fondamentaux Git

Repository



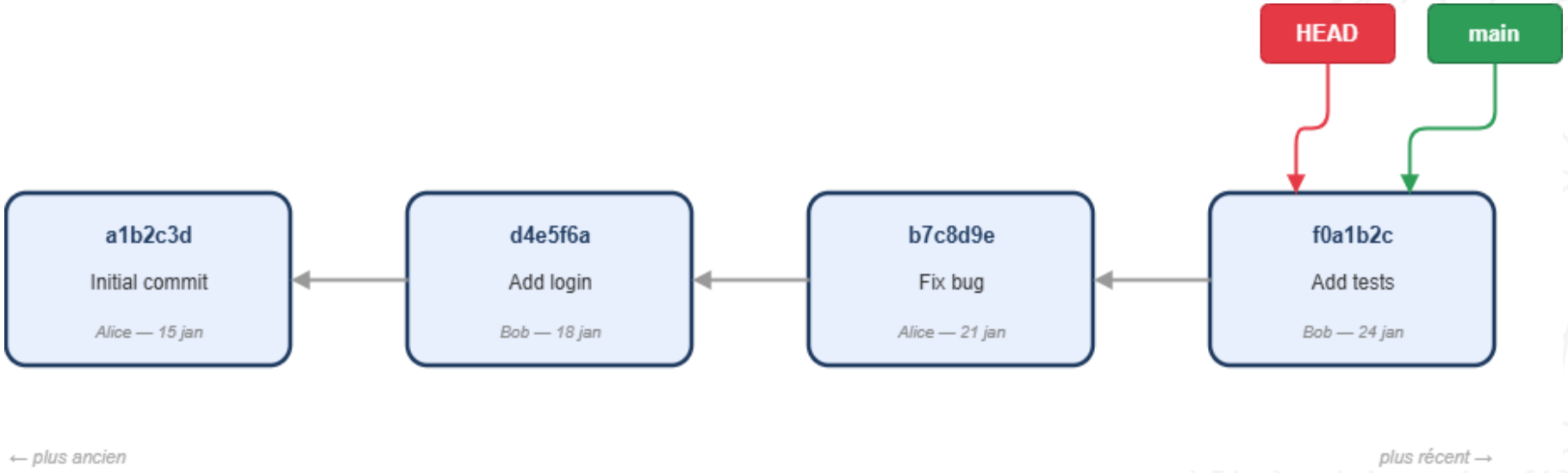


- Un dossier avec un historique caché (.git/)
- Contient tous les commits, branches, tags
- Local : sur votre machine
- Remote : sur GitHub, GitLab, Bitbucket…
- Clone : copie complète d'un remote vers local

Commit : une photo du code

- Snapshot complet du projet à un instant T
- Hash SHA-1 unique (ex: a1b2c3d)
- Message descriptif (obligatoire)
- Auteur + date
- Pointe vers le(s) commit(s) parent(s)
- Immutable : on ne modifie jamais un commit

- Question : Si le commit est immutable, la branche main du repo est-elle protégée de toute attaque ?



Git = Base de données

- Git est une base de donnée
- Une entrée = Un commit
- Un commit = Une “photo” de tout le code à un instant donné
  - pas les changements effectués, l’état complet de chacun des fichiers
  - Les changements effectués sont calculés au vol
- Dans un commit (sha a1b2c3d) :
  - Snapshot
  - Metadata : Qui, quand, commit message
  - Parent : Pointeur vers le dernier commit hash (liste chainée)
    - Cas special : Un commit de merge pointe vers deux parents

Git = DAG (Graphe acyclique dirigé)

- Insérer DAG de commits et un avec deux parents
- Expliquer DAG (no loops; relations are ordered)
- Force de git : Aller voir l'état du projet à n'importe quell commit

Staging area : la zone de préparation

Git a 3 endroits où stocker le code

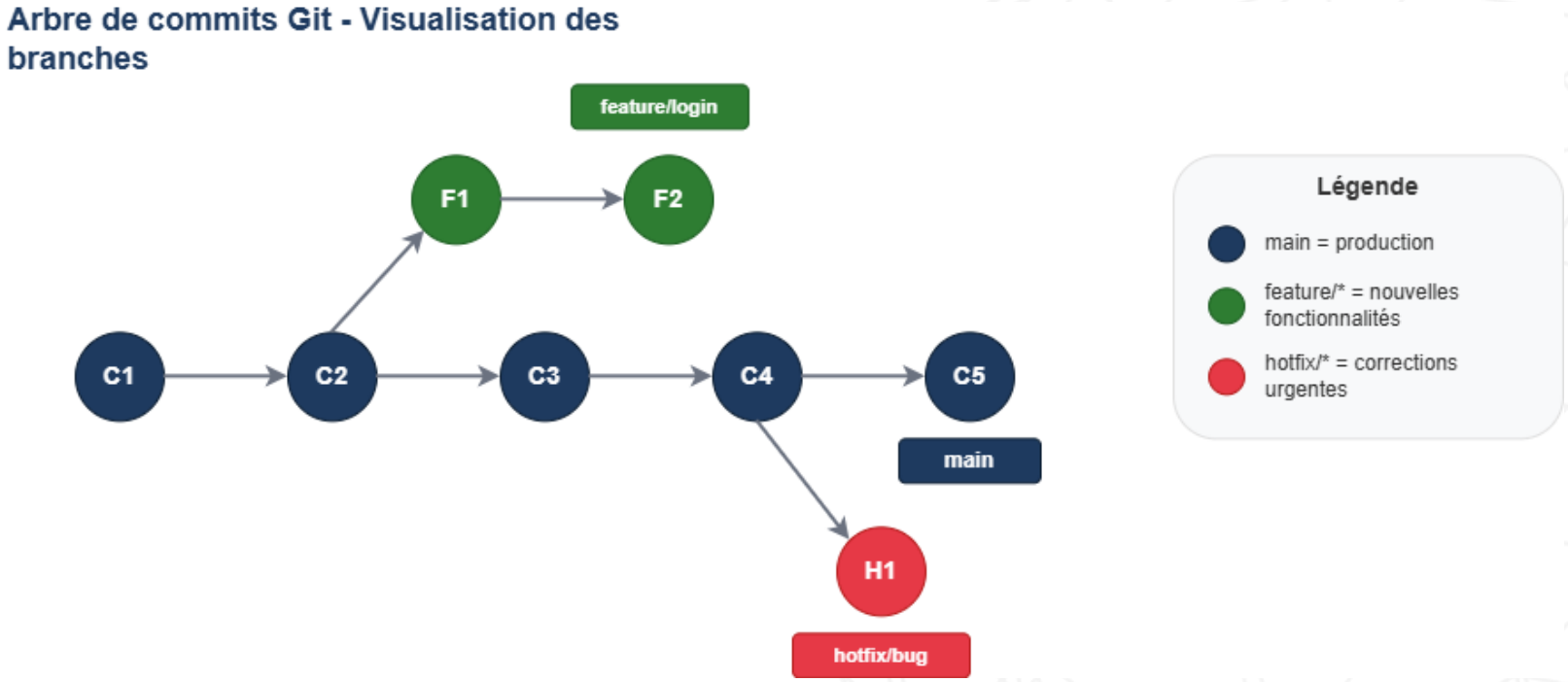
- Répertoire courant : ce qu’on voit dans vscode
- Staging area : git add . -> ce qui est prêt à être commit
  - *Le staging permet de choisir précisément ce qu'on commit*

- Repository : Base de données des commits
  - C’est le repo local, il faut git push pour synchroniser avec le repo distant

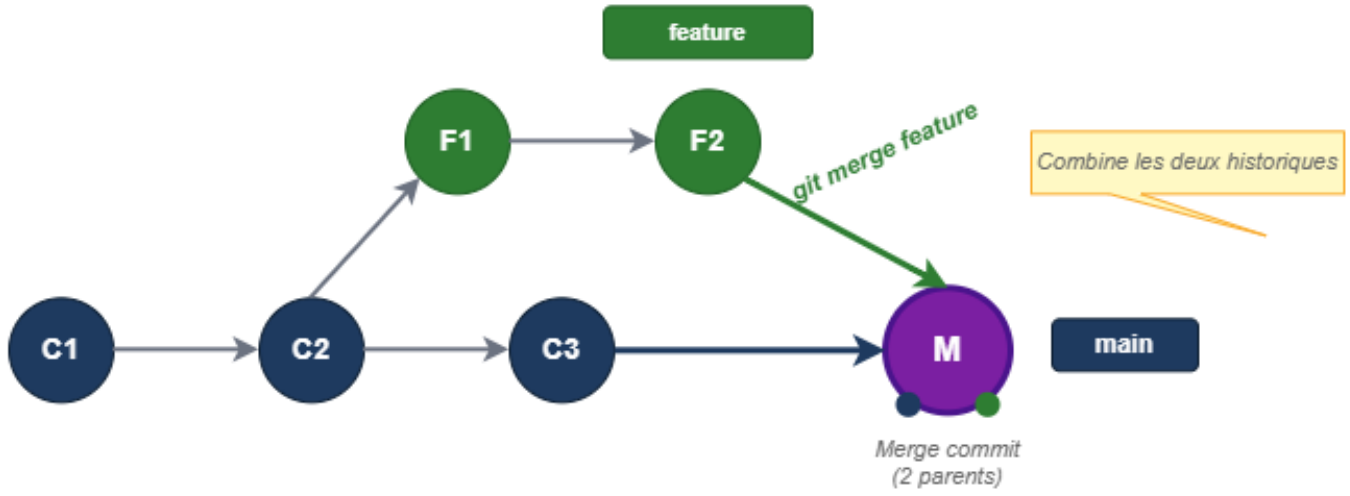
```
# Fichiers modifiés (working directory)
$ git status
  modified: app.py
  modified: README.md

# Ajouter au staging (index)
$ git add app.py

# Créer le commit
$ git commit -m "Fix login bug"
```

Git Branches :

- Une branche = Juste un fichier qui contient un pointeur
  - Pour ça que créer une branche est rapide : Pas de copie de code
- Contient juste 1 info : Le hash du commit
  - Permet de travailler en parallèle
  - main/master : branche principale (prod)
  - feature/* : nouvelles fonctionnalités
  - hotfix/* : corrections urgentes



Merge : fusionner les branches



Comparaison des types de merge

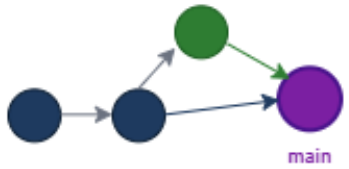
Fast-forward

Pas de divergence, avance simple du pointeur

→ Ligne droite

3-way Merge

Divergence = merge commit avec 2 parents



→ Forme en Y inversé

Quand utiliser ?

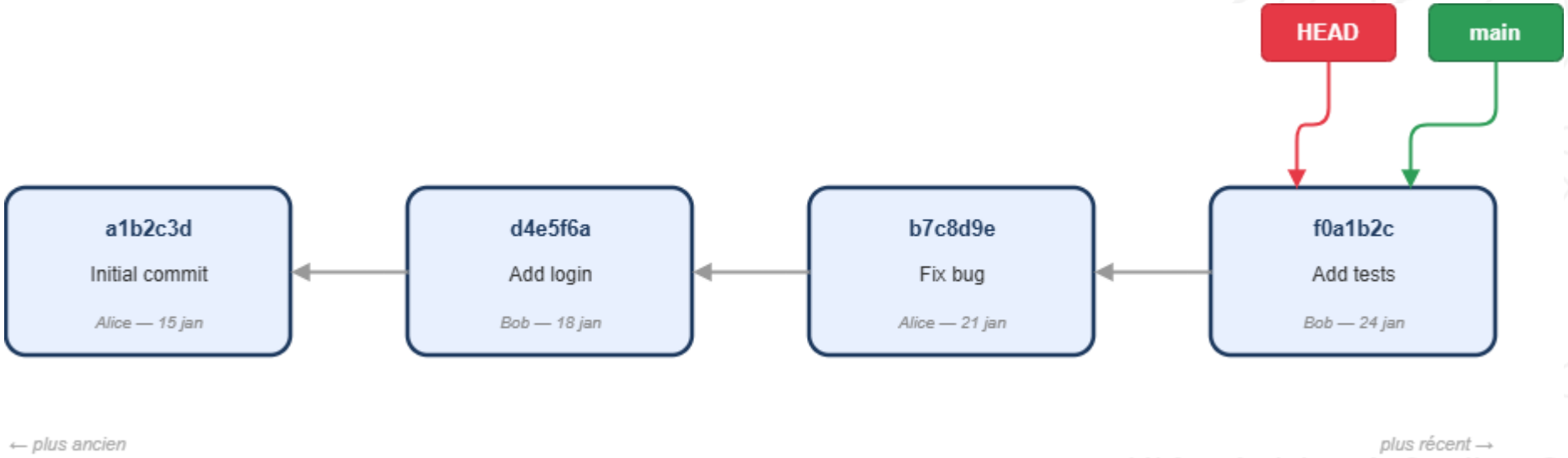
Fast-forward
Quand main n'a pas avancé

3-way merge
Quand les deux branches ont des commits différents

Git checkout

Quelques commandes :

- Git checkout : Déplacer sur une branche ou un commit
  - › Juste HEAD bouge, 0 modification du repo. On scrute juste

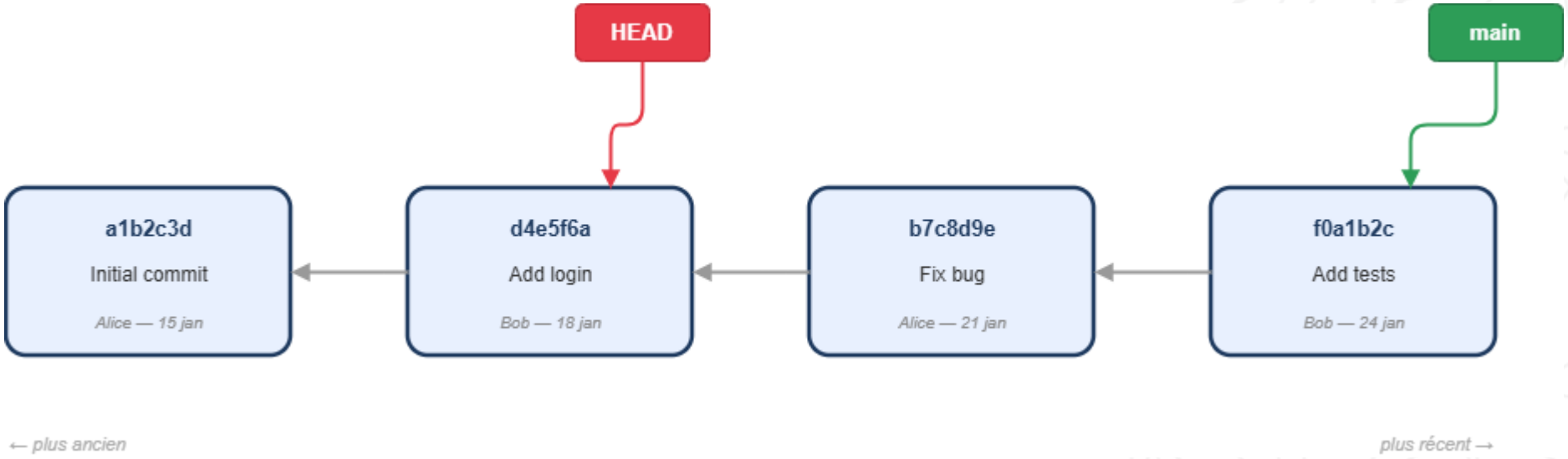


Git checkout

Quelques commandes :

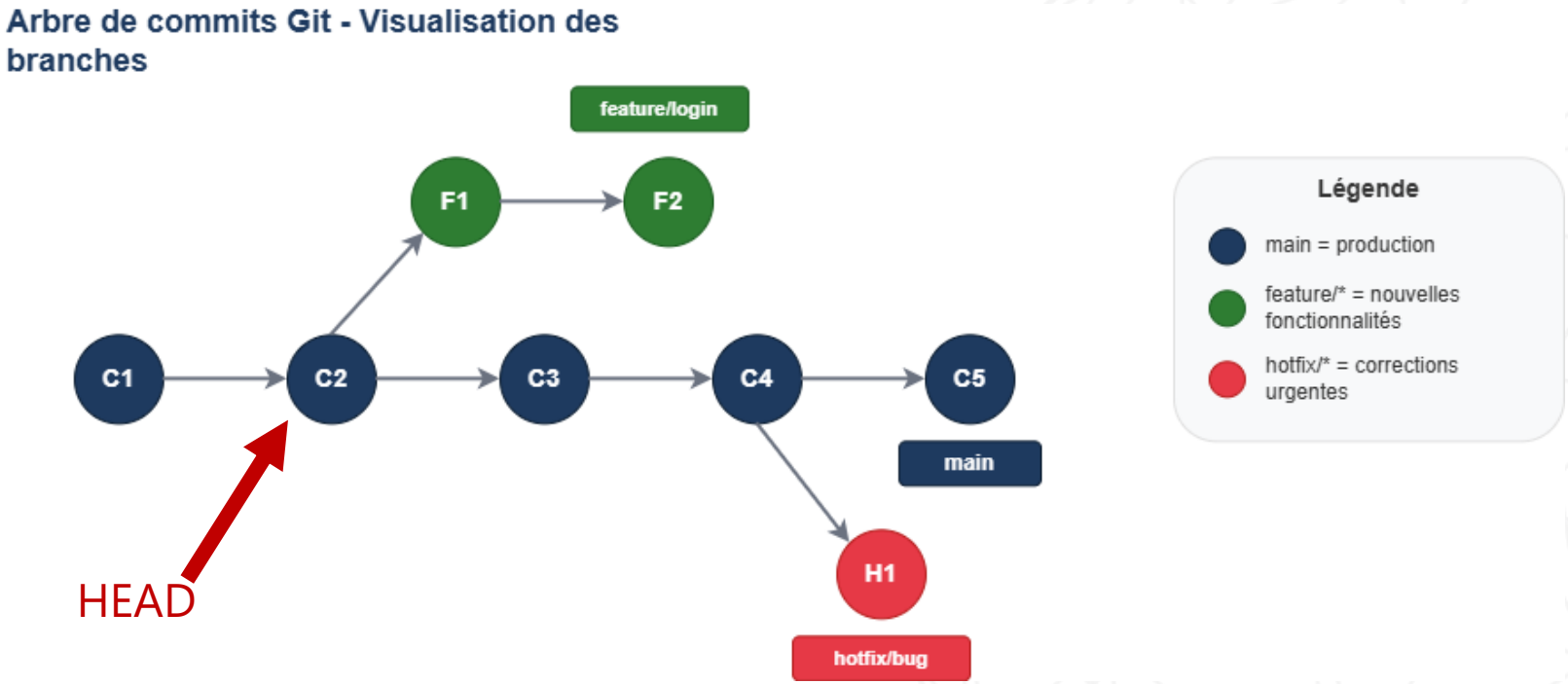
- Git checkout : Déplacer sur une branche ou un commit
  - › Juste HEAD bouge, 0 modification du repo. On scrute juste

Git checkout HEAD^^



Git HEAD

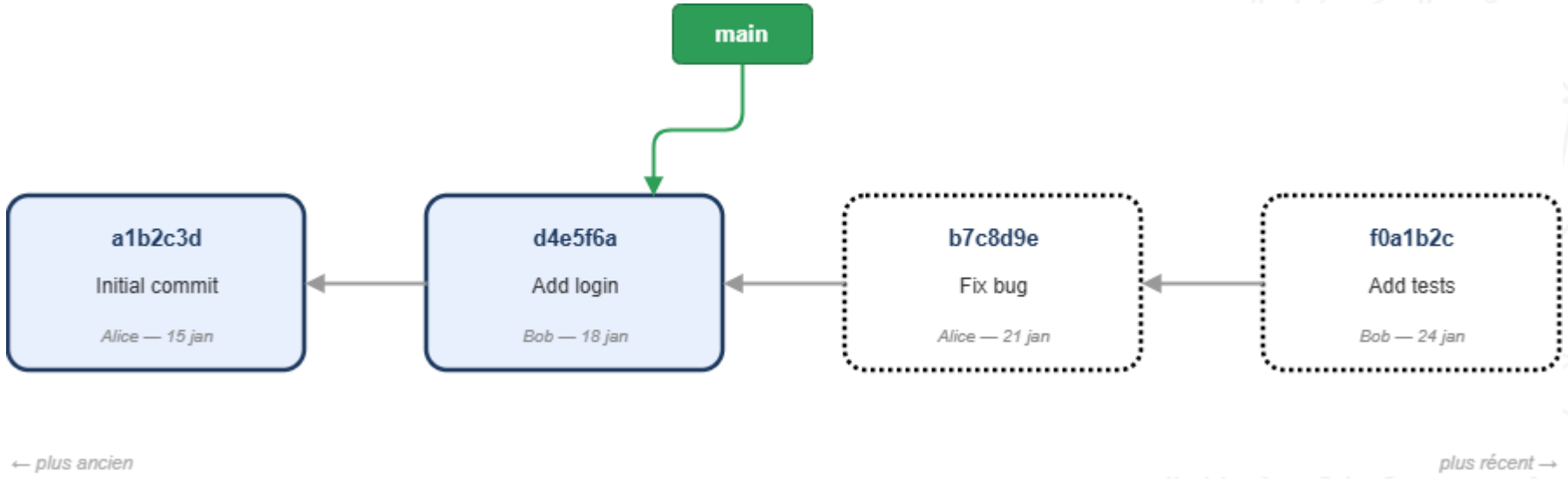
- HEAD : Où on est
  - HEAD -> main (ou branche) -> commit d45af123
  - Git checkout -> moving main
  - Git checkout HEAD^2
- Mode detached : HEAD ne pointe pas vers une branche
  - Commit en mode detached : Possible mais 0 branche ne pointe vers le commit
    - Il sera nettoyé par le garbage collector (30-90j)



Git reset

Quelques commandes :

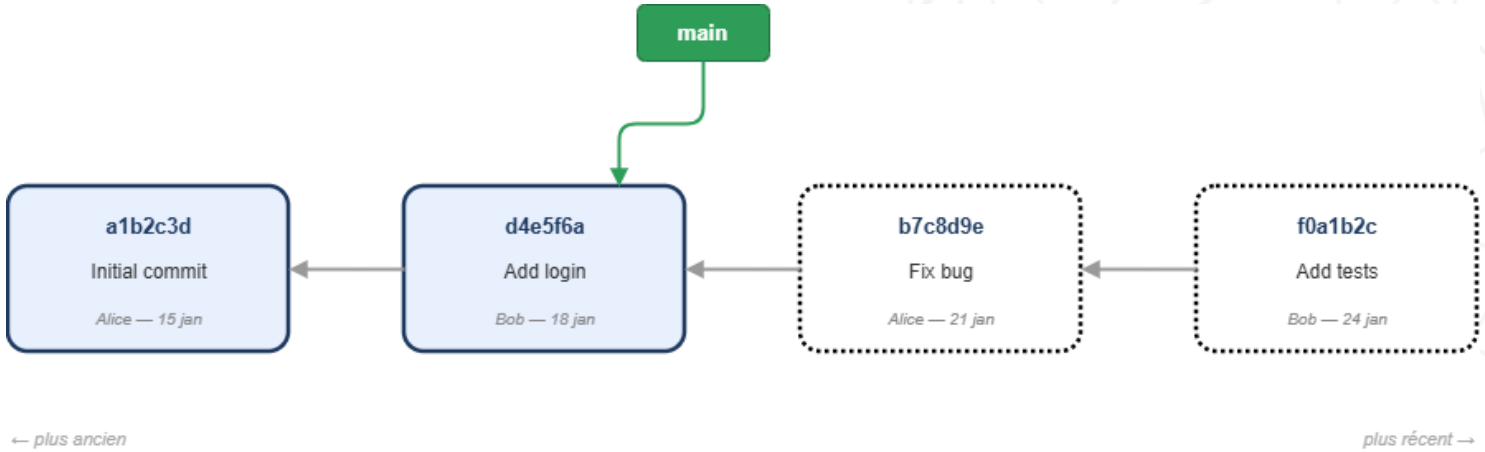
- Git checkout : Déplacer HEAD (sur une branche ou un commit)
  - Juste HEAD bouge, 0 modification du repo. On se ballade juste
- Git reset : Déplacer Une branche
  - Git reset –soft :
    - Déplace juste la branche. Pas de modif pour staging et répoertoire courant
    - Utile quand on a fait 3 commits qu'on veut commit en 1 seul. Faire git reset, puis commit tout d'un coup



Git reset

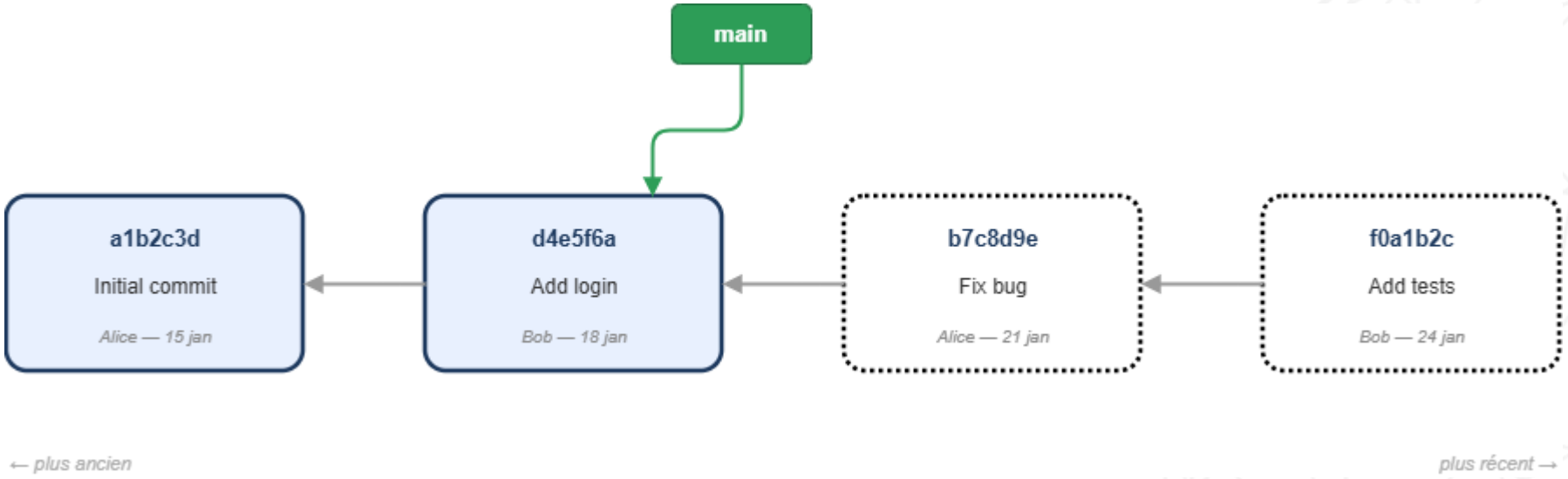
Quelques commandes :

- Git checkout : Déplacer HEAD (sur une branche ou un commit)
  - Juste HEAD bouge, 0 modification du repo. On se ballade juste
- Git reset : Déplacer Une branche
  - Git reset –soft :
    - Déplace juste la branche. Pas de modif pour staging et répoertoire courant
    - Utile quand on a fait 3 commits qu'on veut commit en 1 seul. Faire git reset, puis commit tout d'un coup
  - Git reset --mixed (defaut) :
    - Déplace la branche, nettoie le staging. Les modifs sont encore là, mais unstaged
    - Utile quand on veut split en plusieurs commits des modifications



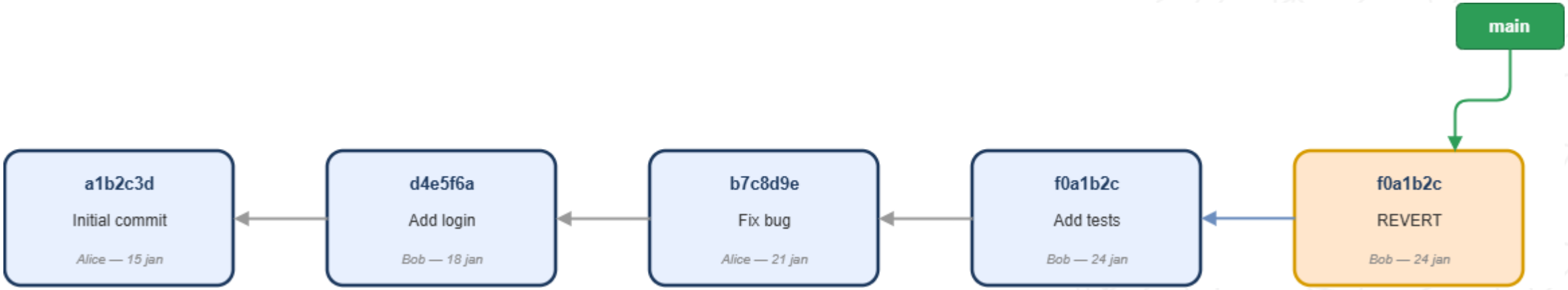
Git reset

- Git reset --hard
  - Déplace la branche, NETTOIE STAGING ET REPERTOIRE COURANT.
  - Tous ces commits sont maintenants orphelins
  - On peut les récupérer mais c'est dur



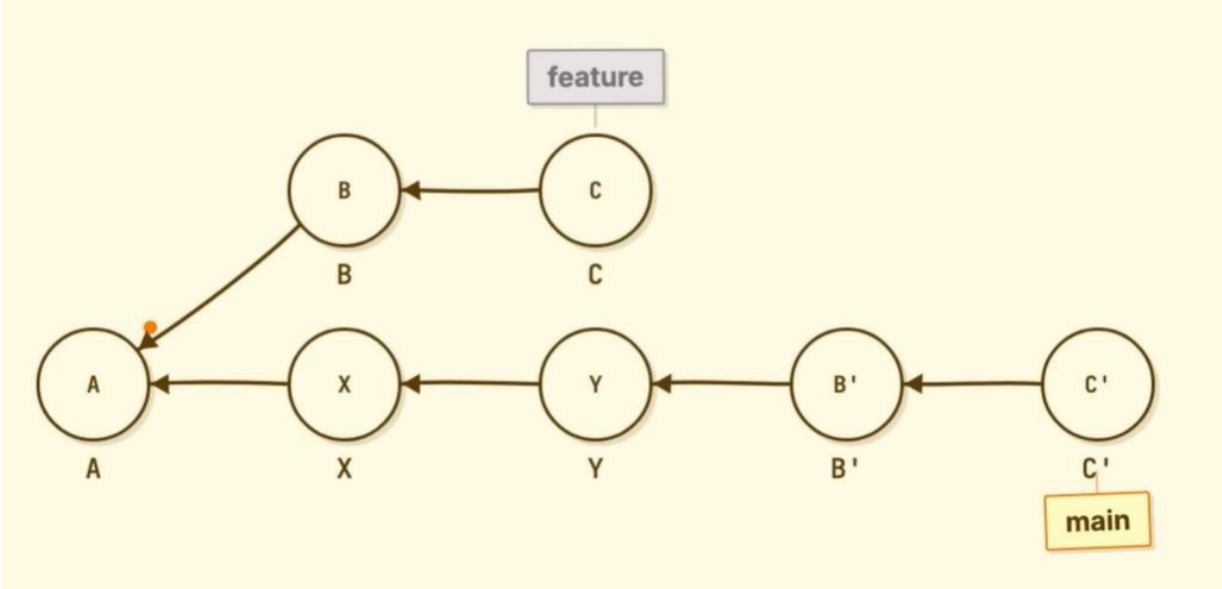
Git revert

- Git revert : Crée un commit qui annule l'ancien
  - Tout l'historique est conservé, aucun commit supprimé
  - Utile pour annuler un commit déjà poussé et partagé
    - Car on ne peut pas réécrire l'historique git



Git rebase

- Git rebase : Rejoue les commits de la branche sur main
- Git rebase « réecrit l'historique »
  - On change le parent de B donc B' apparait, idem pour C'
  - B' et C' ne sont pas les commits originaux, ils ont leur propre hash
  - Mêmes changements mais nouveau commit
- Ne jamais rebase les commits qu'on a push (conflit d'historique)
  - Juste le faire sur nos commits locaux



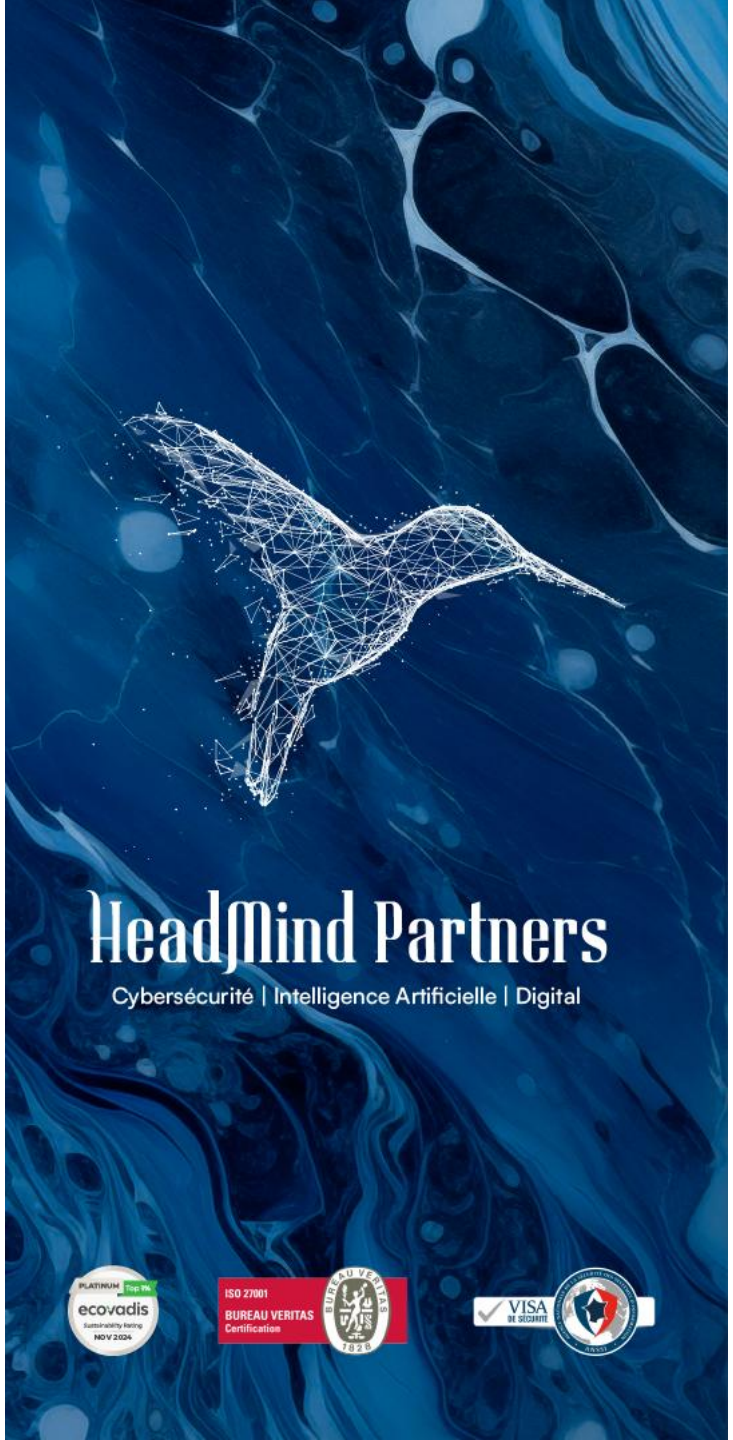
Git reflog : En cas d’urgence

- En cas de git reset --hard mal effectué, commit sur HEAD detached

```
$ git reflog
a1b2c3d HEAD@{0}: checkout: moving from feature to main
b2c3d4e HEAD@{1}: commit: fix critical bug
c3d4e5f HEAD@{2}: commit: update styles
d4e5f6g HEAD@{3}: checkout: moving from main to feature
e5f6g7h HEAD@{4}: commit: initial setup
```

- Une fois le hash trouvé, faire :
  - Git branch recovery e5f6g7h

Accessible pendant 30/90 jours

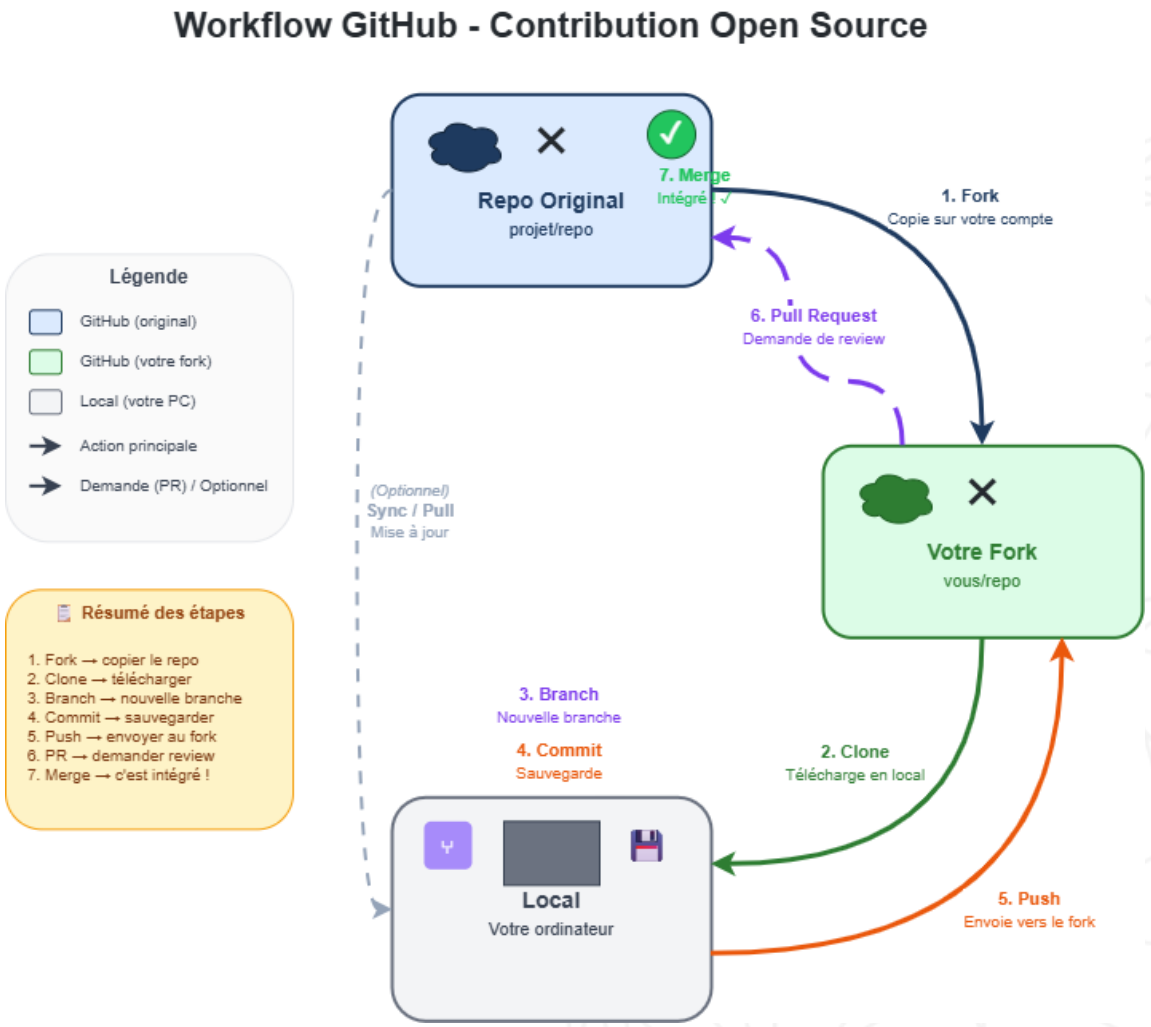


4. Workflow collaboratif

Flow GitHub

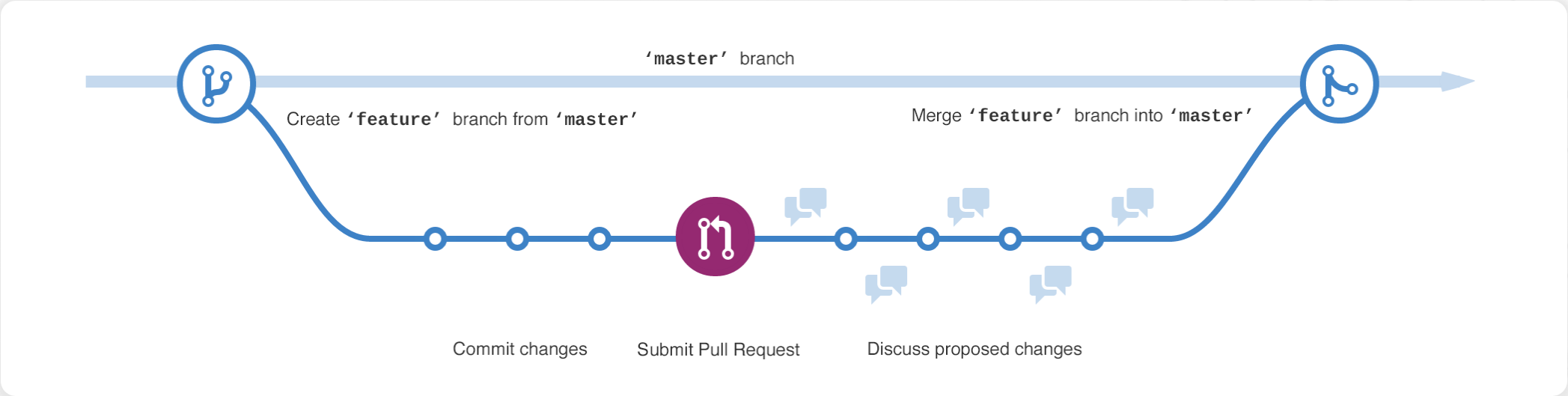
1. Fork : copier le repo sur son compte
    1. Uniquement pour de l'open source.
    2. Pour des projets sur lesquels on est collaborateurs, juste cloner le repo
2. Clone : télécharger en local
3. Branch : créer une branche pour son travail
4. Commit : sauvegarder ses modifications
5. Push : envoyer sur GitHub
6. Pull Request : demander la review
7. Merge : intégrer après validation

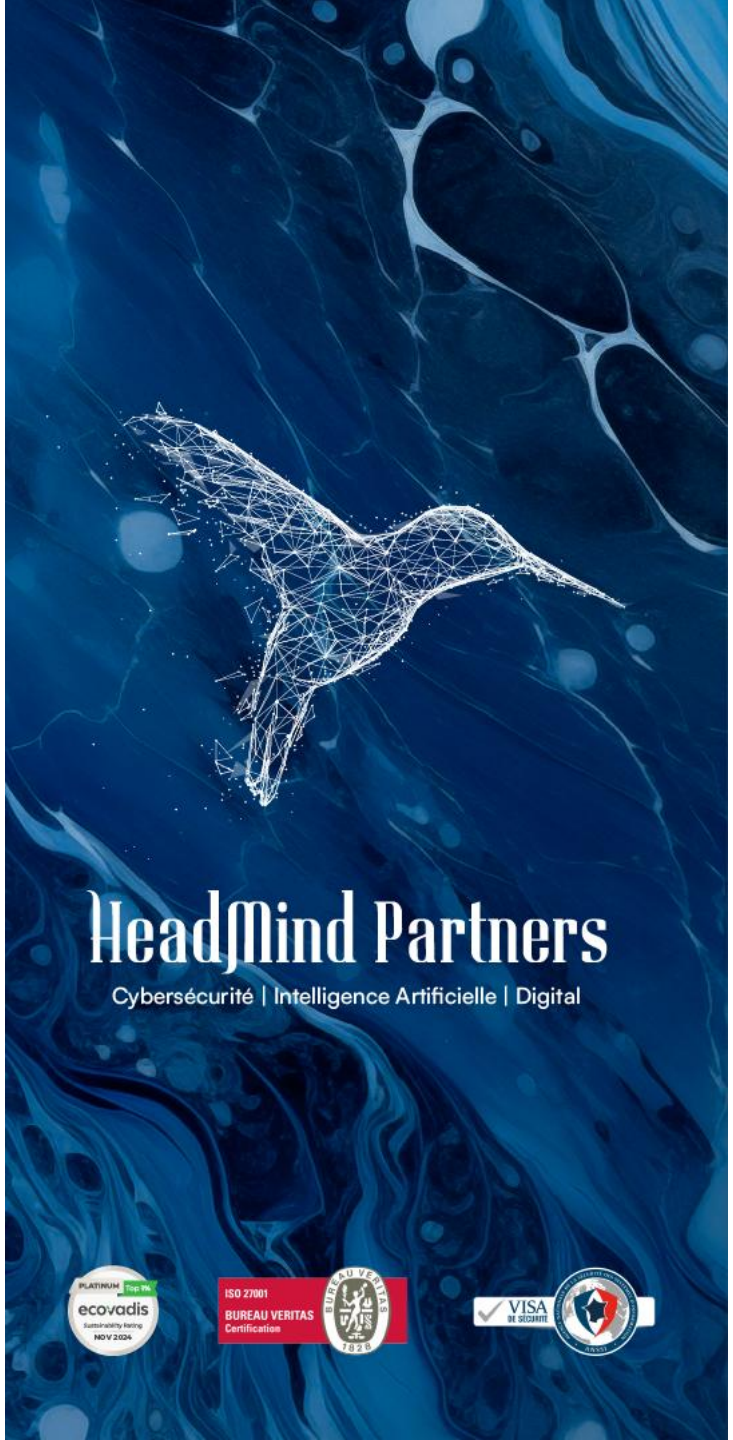
Questions : quelle est la difference entre un Fork et un clone ? Entre une Merge Request et une Pull Request ?



Pull Request (PR)

- Demande d'intégration de sa branche
- Permet la review de code par les pairs
- Discussion et commentaires sur le code
- Vérifications automatiques (CI)
- Historique de toutes les modifications
- Traçabilité : qui a approuvé quoi





5. Investiguer une erreur avec Git

Git log : Historique

```
$ git log --oneline
a1b2c3d Fix login bug
e4f5g6h Add user authentication
i7j8k9l Initial commit

$ git log --oneline --graph --all
* a1b2c3d (HEAD -> main) Fix login bug
| * x1y2z3a (feature/signup) Add signup form
|/
* e4f5g6h Add user authentication
```

Options utiles : --oneline, --graph, --all, --author, --since

git blame : qui a écrit ça ?

```
$ git blame app.py
a1b2c3d (Alice  2024-01-15) def login():
a1b2c3d (Alice  2024-01-15)     user = get_user()
e4f5g6h (music  2024-01-10)     if user = None:  # BUG!
a1b2c3d (Alice  2024-01-15)         return error()
```

Chaque ligne montre : commit, auteur, date

git show : détail d'un commit

```
$ git show e4f5g6h
commit e4f5g6h
Author: music <music@music.music>
Date:   Wed Jan 10 14:30:00 2024

    Simplify error handling

diff --git a/app.py b/app.py
- if user == None:
+ if user = None:  # Oops, = au lieu de ==
```

Montre le diff complet du commit

Astuce : Extension VSCode GitLens

```
27        data = data[data["product"].str.contains(search)]
28
29    # Filtre
30    products = data["product"].unique().tolist()
31    selected = st.multiselect("Filtrer par produit", products, default=products)
32    filtered = data[data["product"].isin(selected)]
33
34    display_metrics(filtered)
35
36    st.subheader("Données")
37    st.dataframe(filtered)
38
39    # Graphique en barres par catégorie
40    st.subheader("Ventes par produit")
41    chart_data = filtered.groupby("product")["amount"].sum()
42    st.bar_chart(chart_data)
43
```

Serpico 4 hours ago *(February 20th, 2026 10:10 AM)*

Add search functionality

45c5930 | Explain | Connect to GitHub... | ...

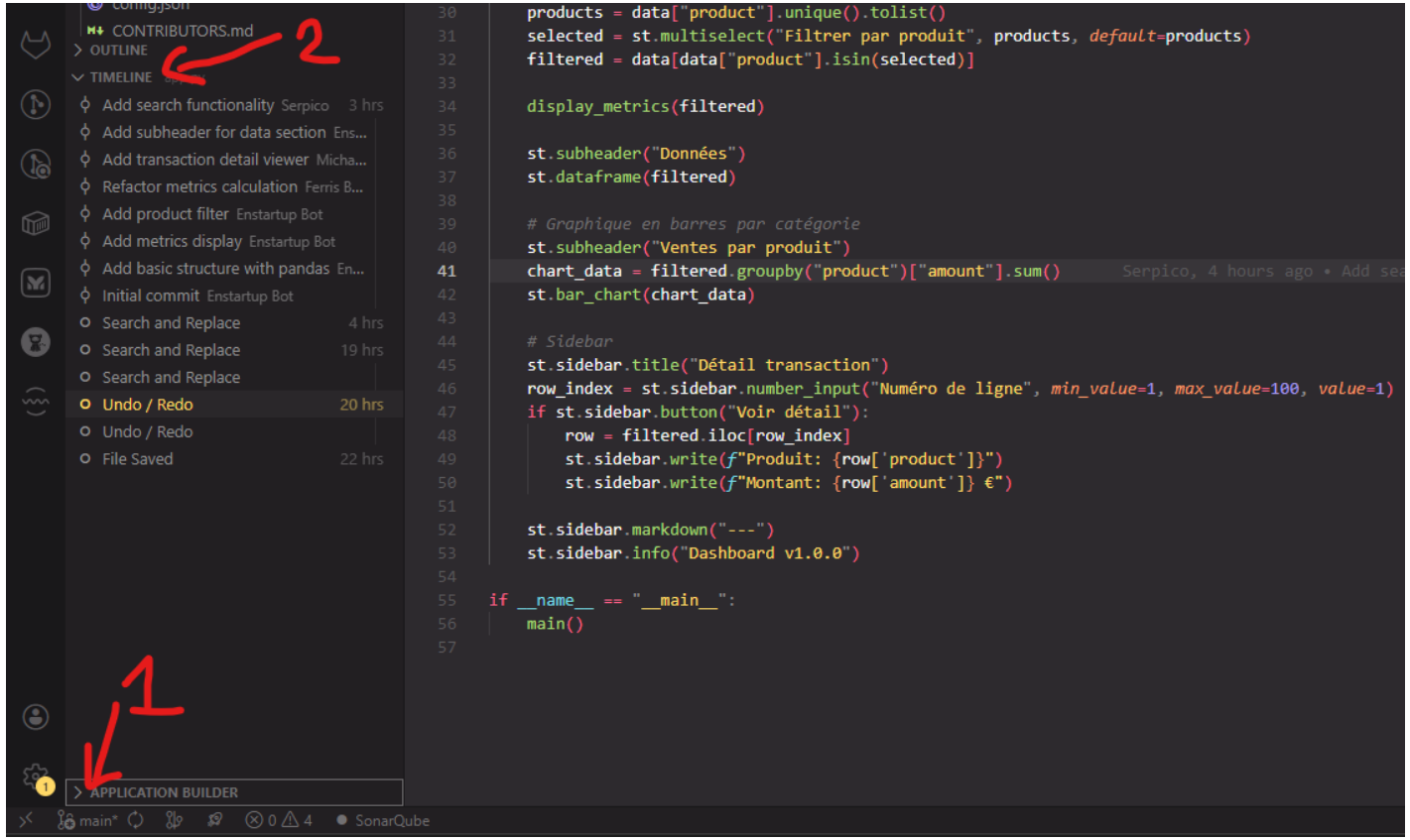
```
+ chart_data = filtered.groupby("product")["amount"].sum()
```

Changes ac7ac0e ↔ 45c5930 |

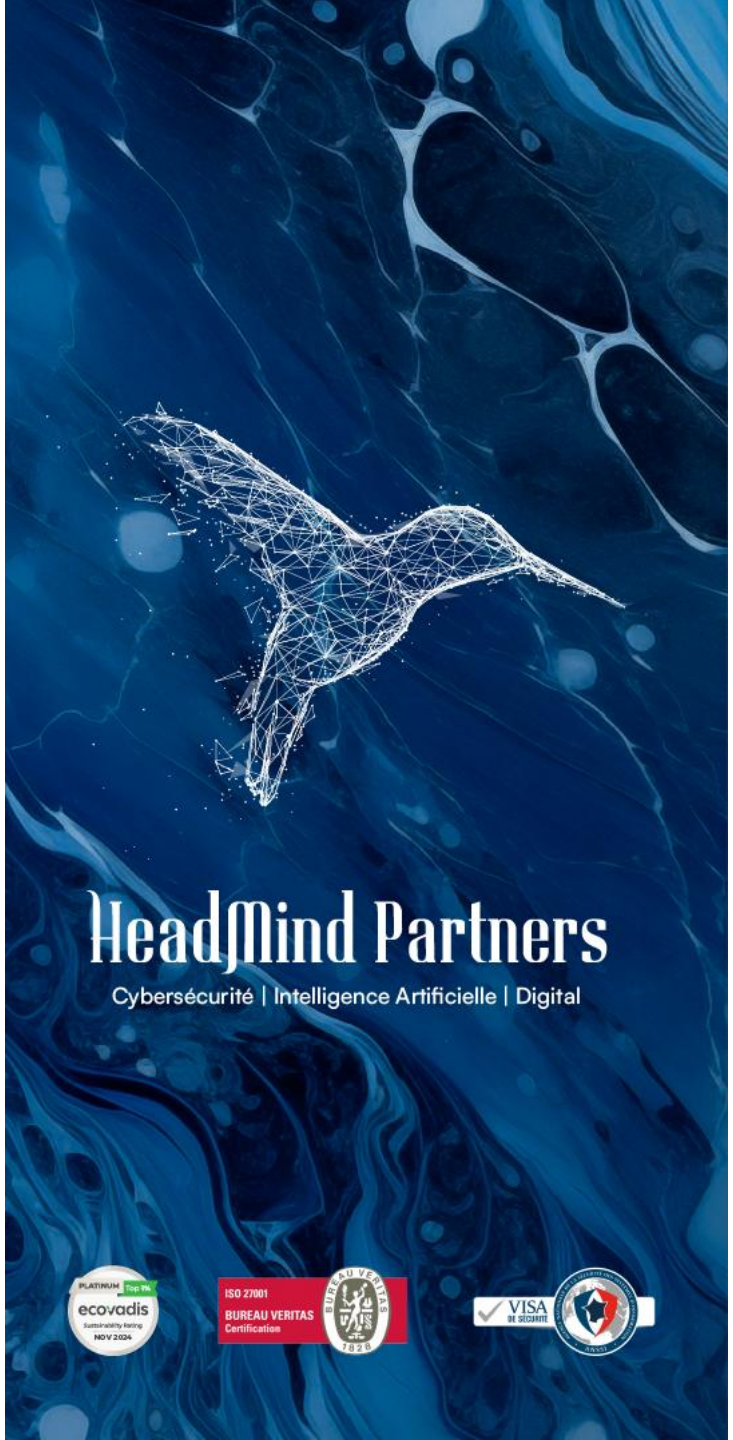
Serpico, 4 hours ago • Add search functionality

GitLens est une extension VSCode qui intègre les informations git dans VSCode

Astuces : VSCode et Git



1. *On peut facilement changer de branche dans VSCode directement*
2. *On peut facilement afficher l'historique et un graphe de commits associés dans VSCode*



6. Workflow Hotfix

Problème : Bug en prod

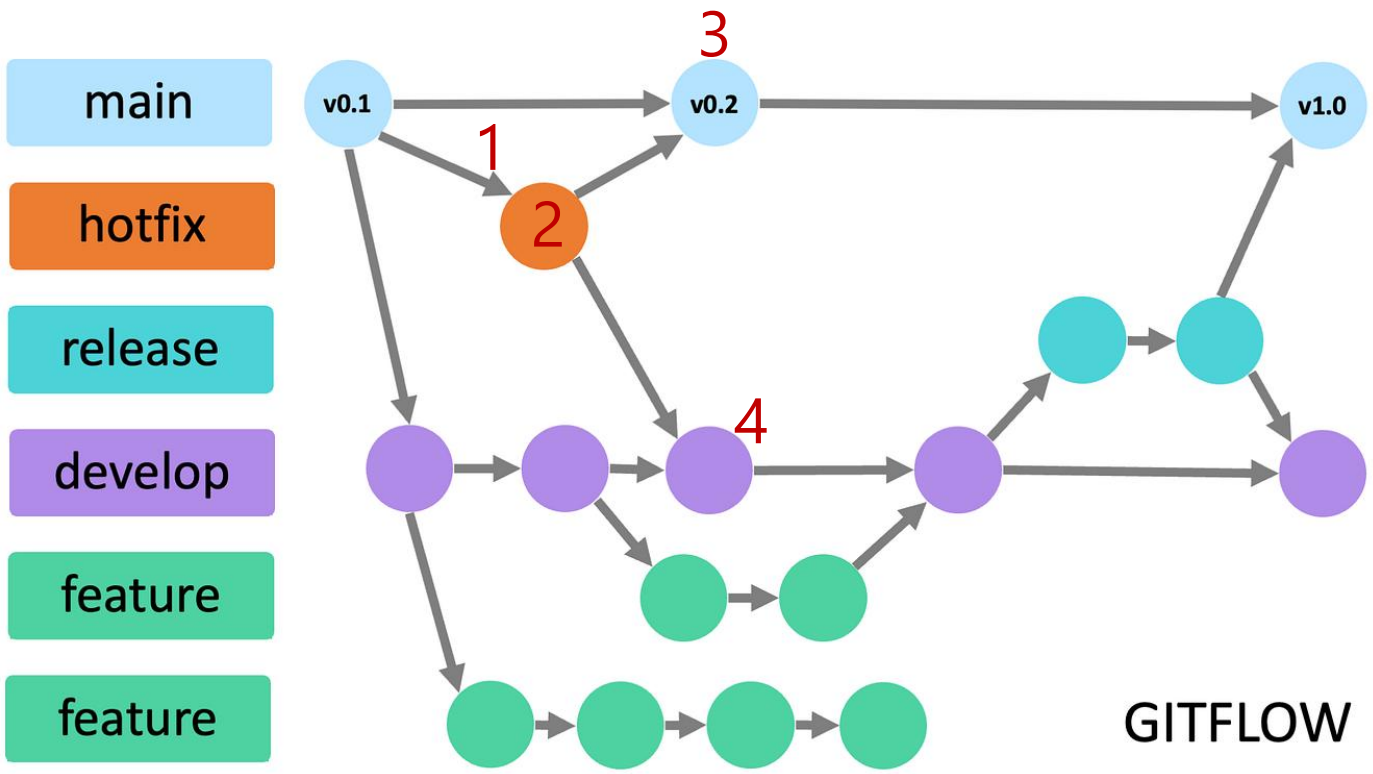
- Bug critique découvert en production
- Il faut le corriger rapidement
- La branche dev contient des features pas prêtes
  - On ne peut pas travailler sur dev puis propager vers main

- Solution : Créer une branche depuis le main “hotfix”, et propager vers prod puis dev

Solution : branche hotfix

1. Créer hotfix/ depuis main (état stable)
2. Corriger le bug
3. Merger hotfix → main
4. Cherry-pick le fix vers dev

Résultat : fix en prod SANS embarquer les features WIP



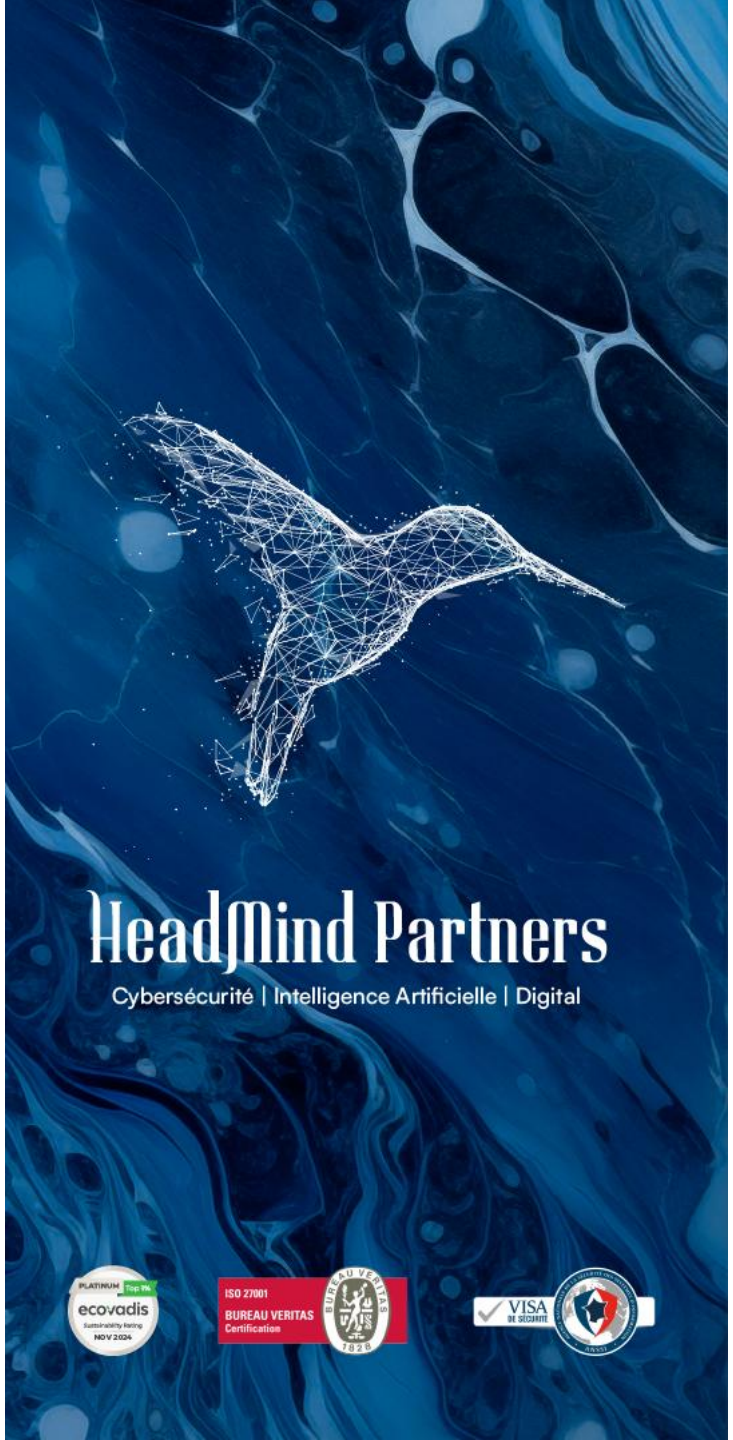
Cherry-pick : appliquer un commit ailleurs

Prend 1 commit spécifique depuis n'importe où et l'applique sur la branche courante. Utile pour :

- **Corriger quand on a hotfix sur une branche feature** par erreur, et on veut propager JUSTE le bugfix sur main
- **Réappliquer un commit perdu** ou annulé
- **Partager un commit précis** entre des branches

```
# Sur main, après le merge du hotfix
$ git log --oneline -3
f1x2b3g Fix critical bug    ← on veut celui-ci
a1b2c3d Previous commit
e4f5g6h Older commit

# Sur dev, appliquer le fix
$ git checkout dev
$ git cherry-pick f1x2b3g
# Le commit est maintenant aussi sur dev
```

Cherry-pick copie un commit spécifique sur la branche courante



7. CI/CD : Concepts

Pourquoi CI/CD ?

1x

Coût d'un bug en dev

10x

Coût d'un bug en test

100x

Coût d'un bug en prod

CI vs CD : A SPLIT EN DEUX

CI — Continuous Integration

- Build automatique à chaque push
- Tests automatiques
- Lint / analyse de code
- Feedback rapide au développeur

Outils associés :

- **pytest / JUnit / Jest** — frameworks de tests selon le langage
- **SonarQube** — analyse de qualité de code et détection de vulnérabilités
- **Trivy / Snyk** — scan de sécurité des dépendances et images Docker
- **Black / ESLint / Flake8** — linters selon le langage

CD — Continuous Delivery/Deployment

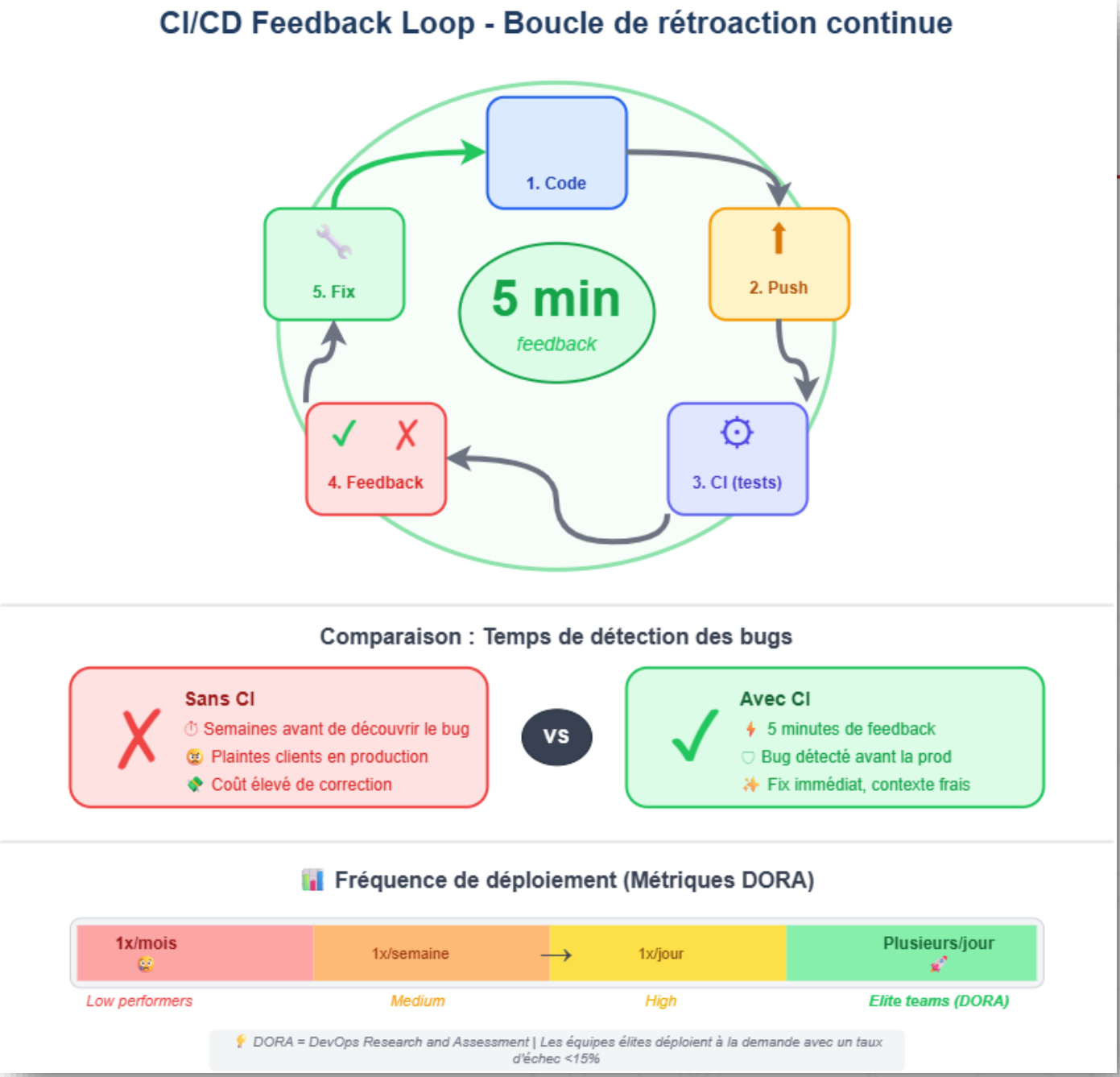
- Delivery : artefact prêt à déployer
- Deployment : déploiement automatique
- Staging → Production
- Rollback automatisé si erreur

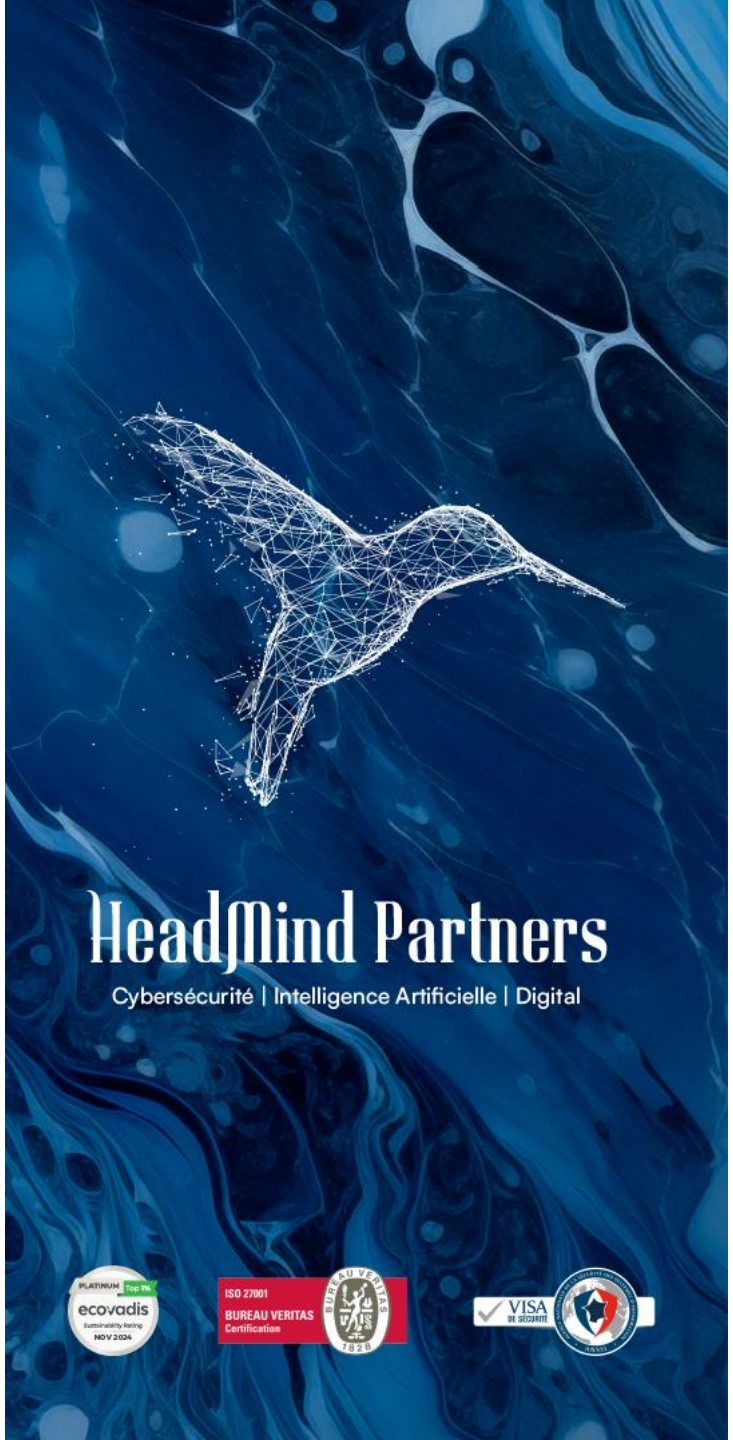
Outils associés

- **Docker / Docker Hub** — construction et stockage des images
- **Kubernetes / Helm** — orchestration des déploiements
- **Terraform / Ansible** — infrastructure as code, provisioning automatisé
- **ArgoCD / FluxCD** — GitOps, synchronisation automatique dépôt → cluster
- **Azure DevOps / AWS CodeDeploy / GCP Cloud Deploy** — pipelines cloud natifs

Le feedback loop

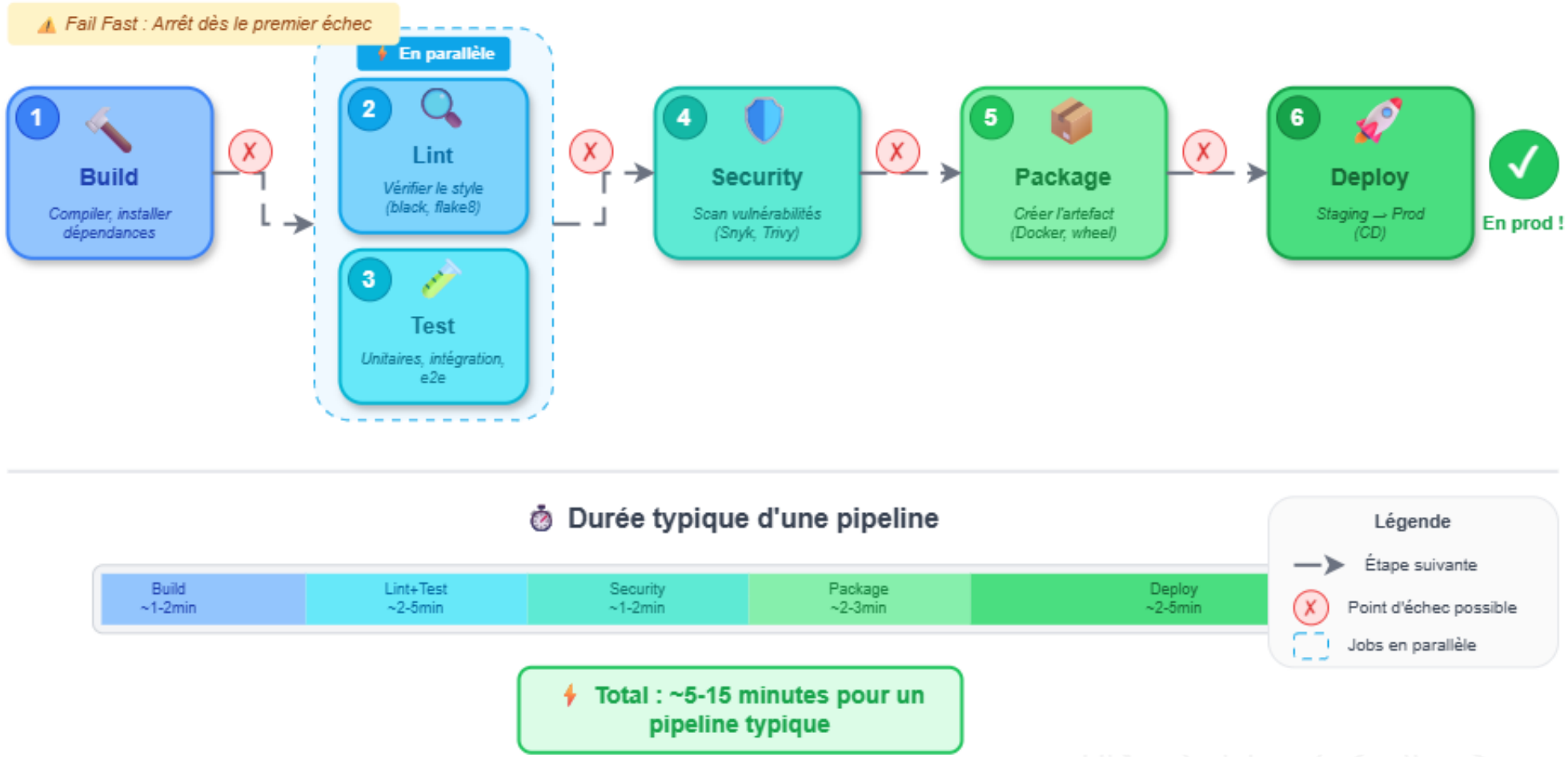
- Plus le feedback est rapide, plus on corrige vite
  - Sans CI : bug découvert en prod après des semaines et des plaintes client
  - Avec CI : bug découvert en 5 minutes après le push
- Elite teams (DORA) : deployer plusieurs fois/jour
- Mauvais score : deployer 1x/mois





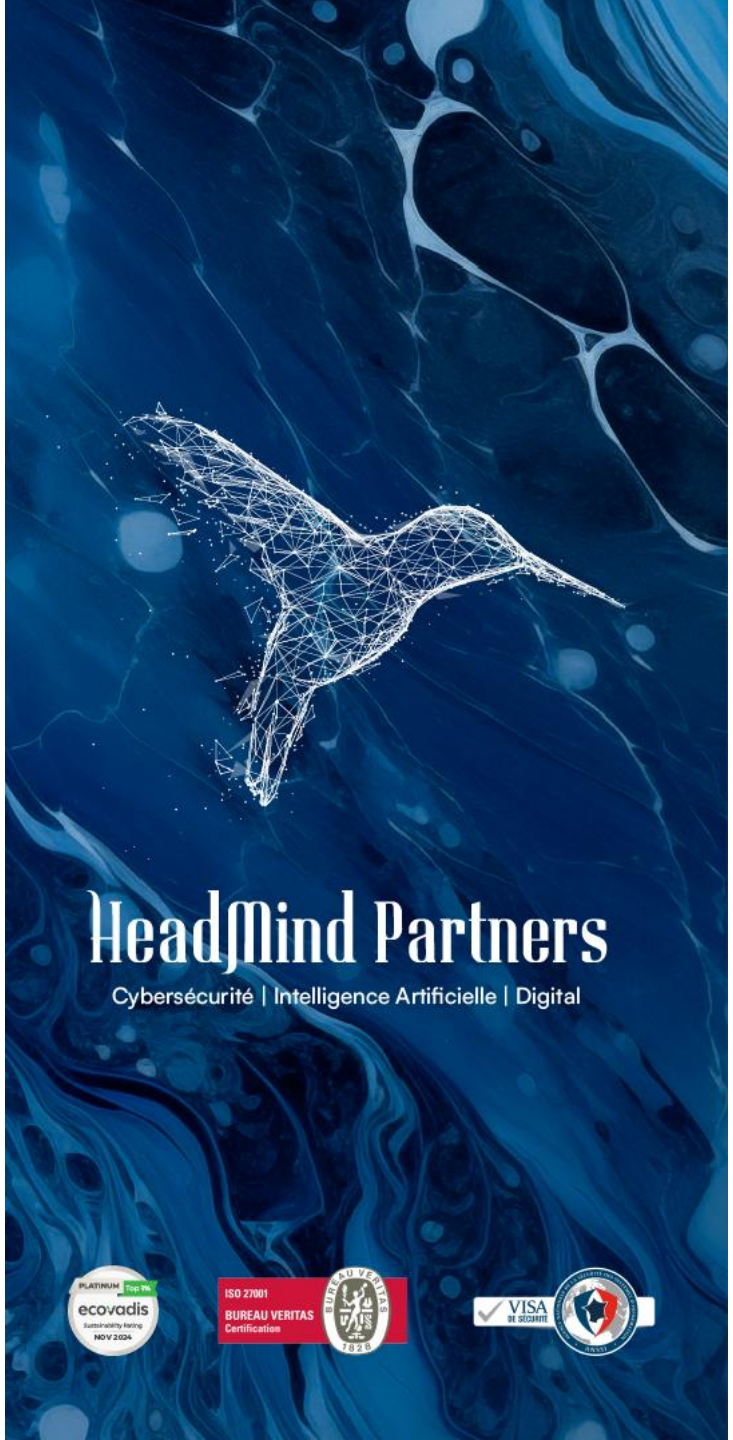
8. Anatomie d’un pipeline CI

Les étapes (stages) d'un pipeline



Principes clés

- Fail fast : arrêter dès qu'une étape échoue
- Parallélisation : lint + test en parallèle
- Cache : ne pas retélécharger les dépendances
- Artifacts : passer des fichiers entre stages
- Environnements : dev → staging → prod



9. GitHub Actions

Structure d’un workflow

```yaml
# .github/workflows/tests.yml
name: Tests

on:
  push:
    branches: [main, dev]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements.txt
      - run: pytest -v
```

1 pipeline = Fichier(s) YAML dans .github/workflows/

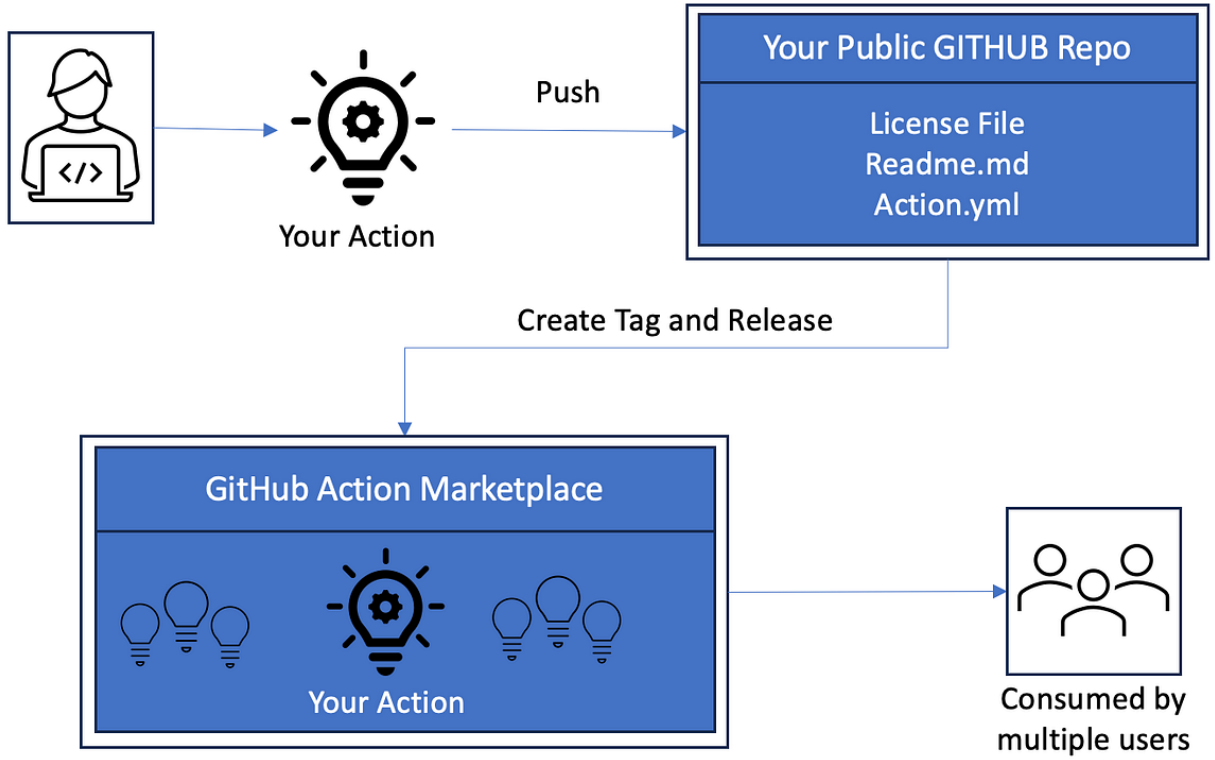
“Triggers” : quand lancer la CI

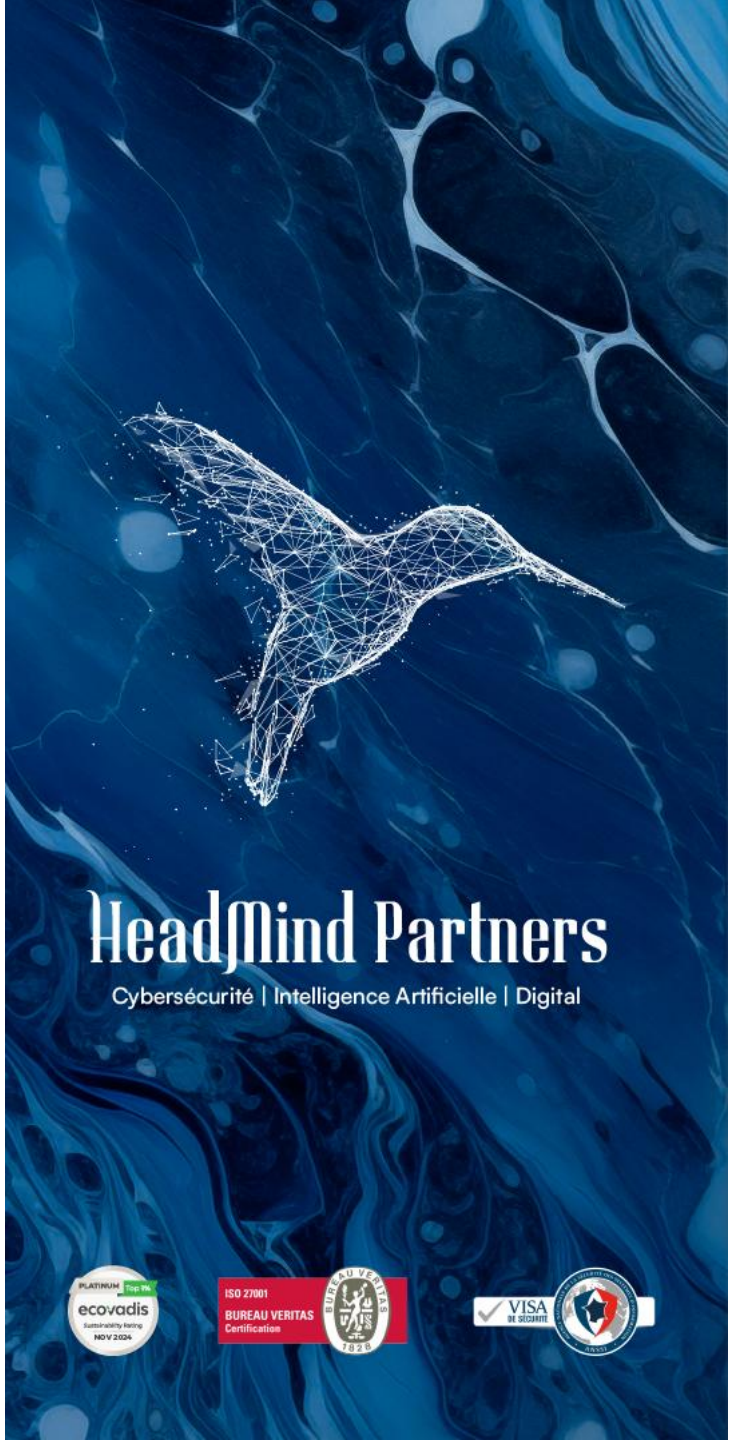
La CI ne tourne pas tout le temps — on définit précisément quels événements déclenchent la pipeline.

- push : à chaque push sur une branche
- pull_request : à l'ouverture/mise à jour d'une PR
- schedule : cron (ex: tests nocturnes)
- workflow_dispatch : déclenchement manuel
- paths : seulement si certains fichiers changent

“GitHub Actions Marketplace”

- Briques réutilisables créées par la communauté
- actions/checkout : récupérer le code
- actions/setup-python : installer Python
- actions/cache : mettre en cache les dépendances
- codecov/codecov-action : upload coverage
- Des milliers d'actions disponibles

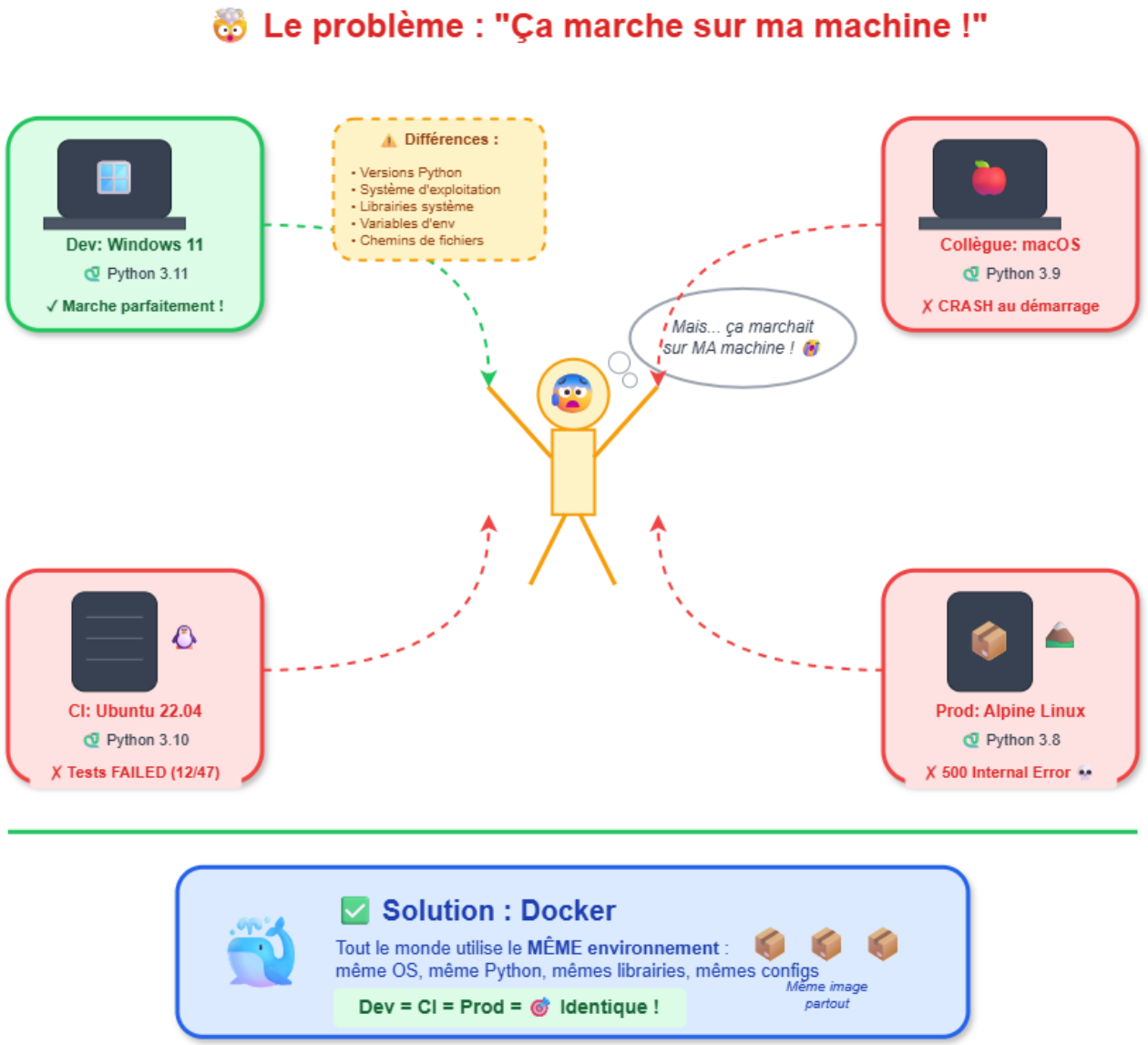




10. Introduction à Docker

10. Introduction à Docker

- Dev sur Windows, collègue sur Mac
- CI sur Ubuntu, Prod sur Alpine Linux
- Versions Python différentes
- Dépendances système manquantes
- Solution : embarquer l'environnement avec le code



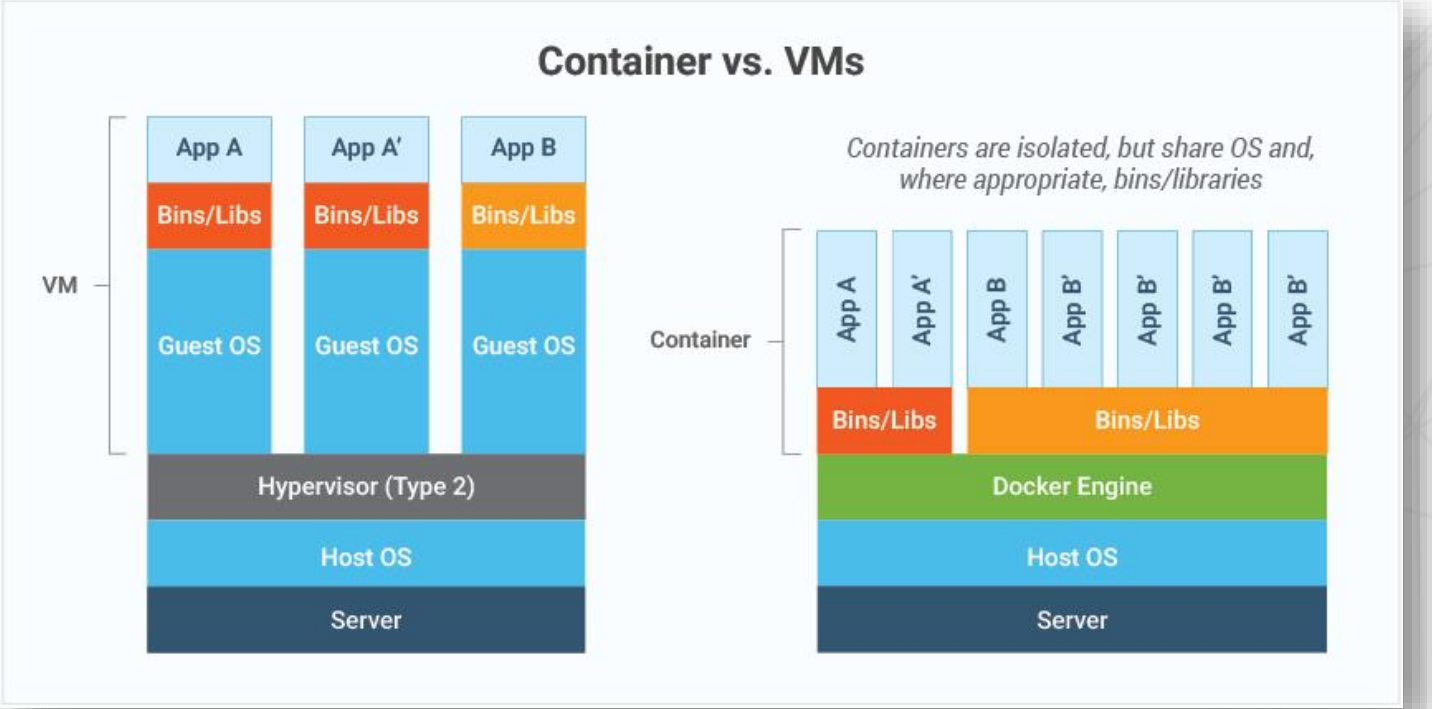
Conteneur vs VM

Machine Virtuelle

- OS complet (Go de RAM)
- Démarrage en minutes
- Isolation forte
- Overhead important

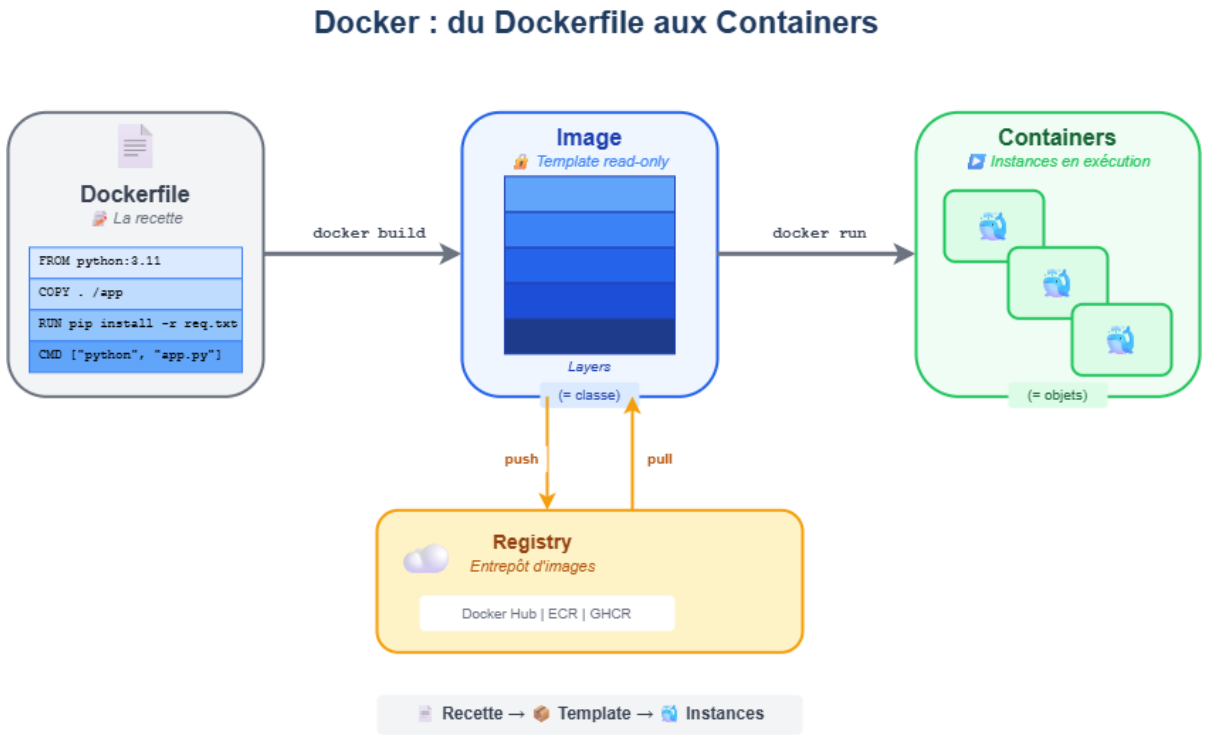
Conteneur

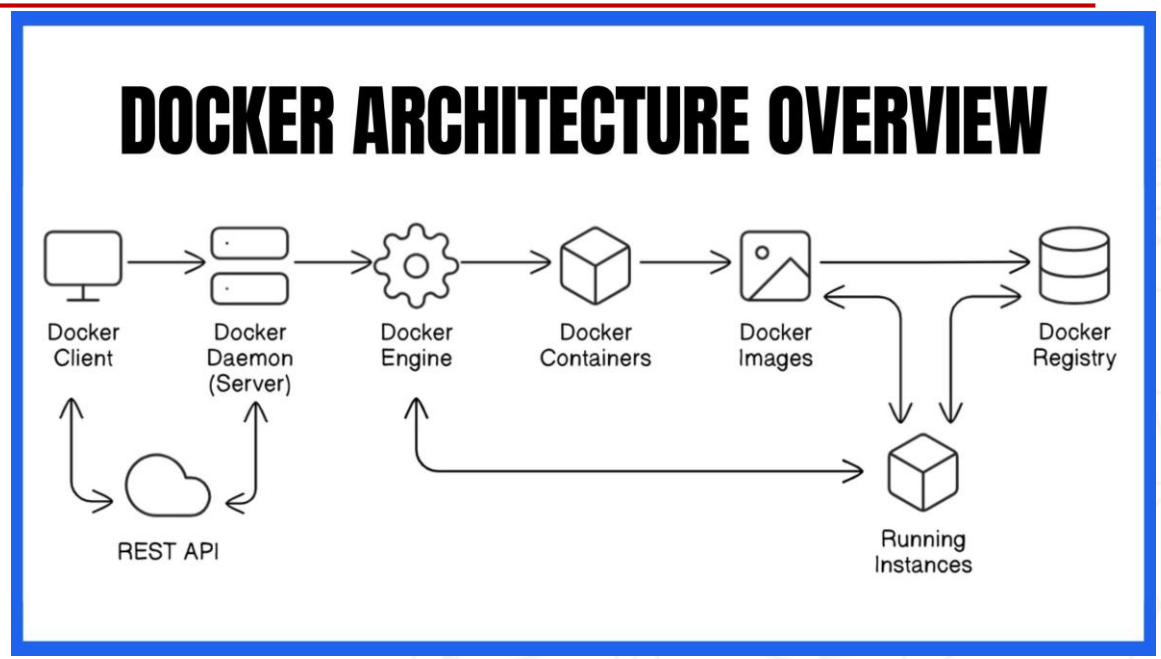
- Partage le kernel de l'hôte
- Démarrage en secondes
- Isolation légère
- Très efficace en ressources



Concepts Docker

- Image : template read-only (comme une classe)
- Container : instance d'une image (comme un objet)
- Dockerfile : recette pour construire une image
- Registry : stockage d'images (Docker Hub, ECR)
- Layer : chaque instruction = une couche cachée



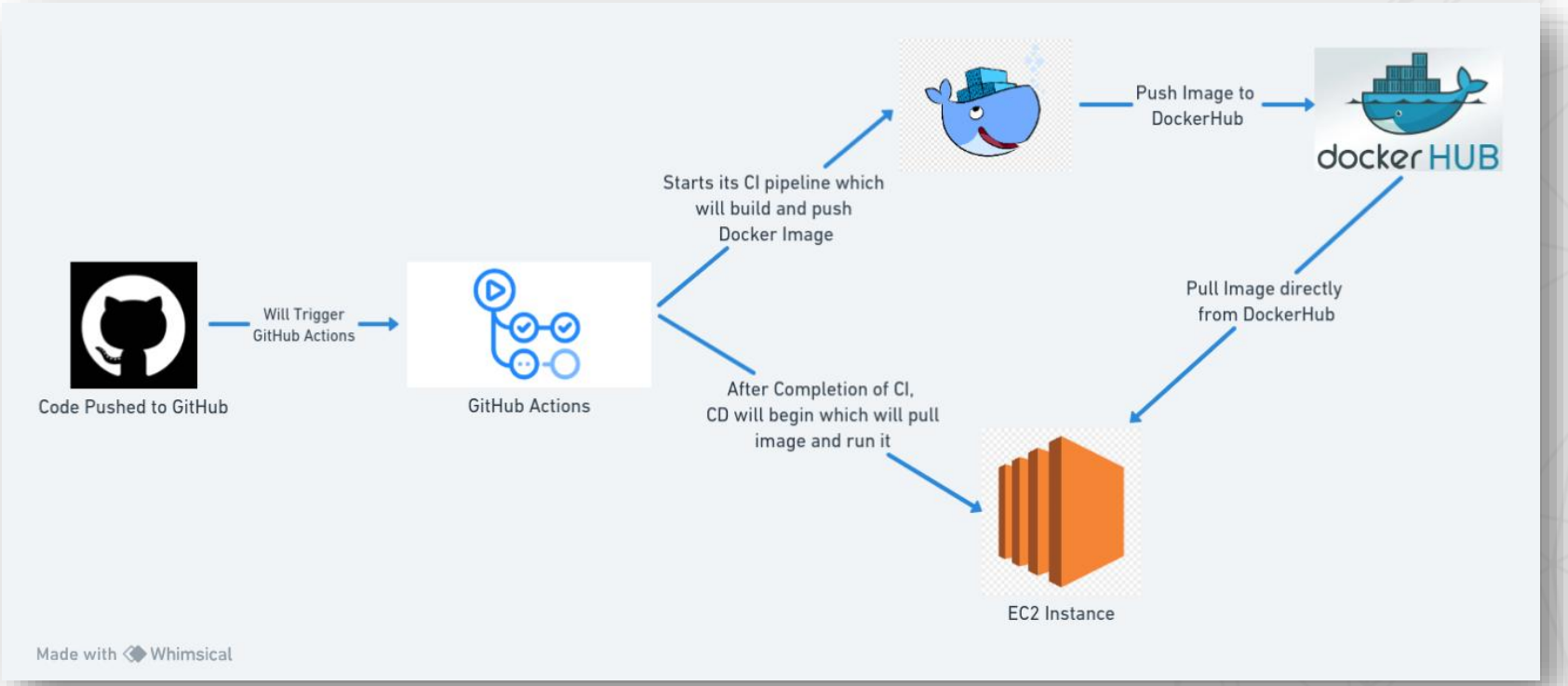


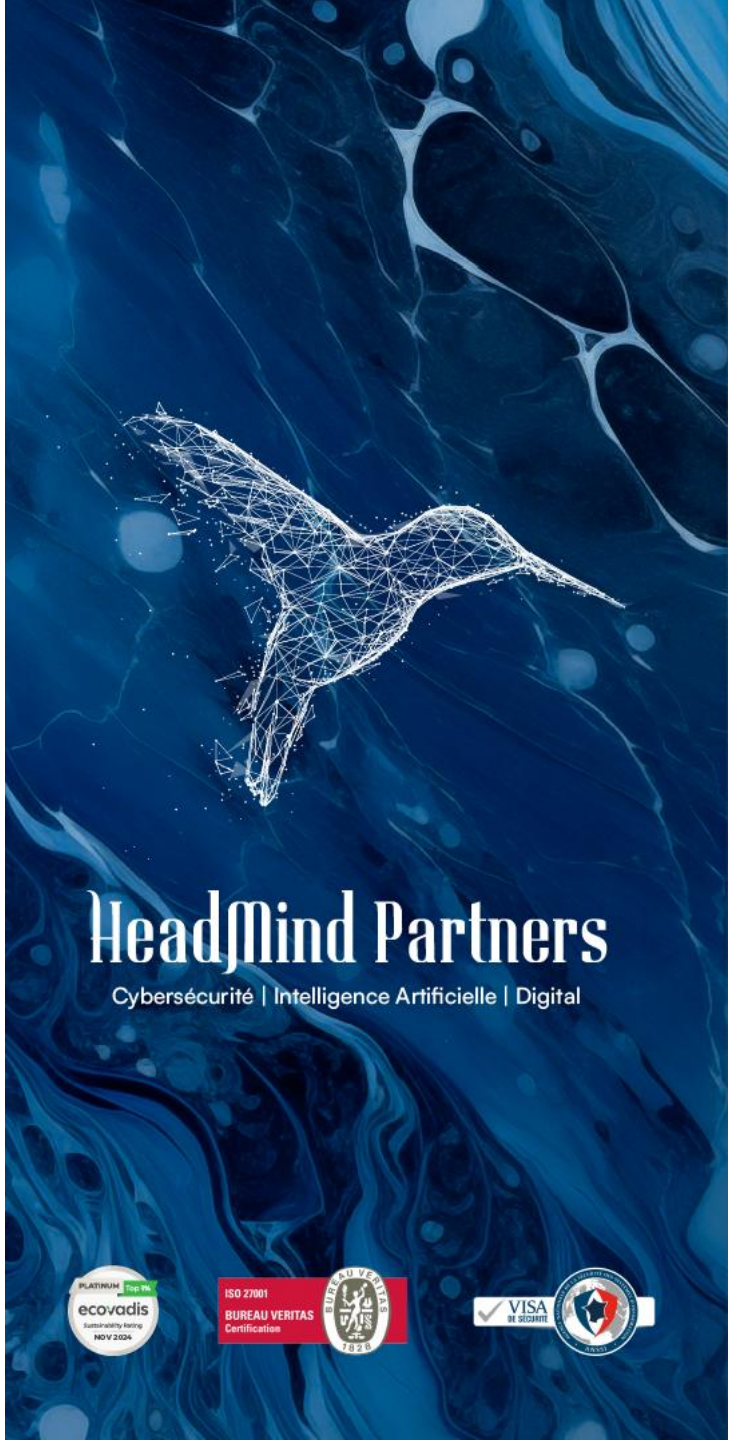
Avantages de Docker

- Reproductibilité : même image partout
- Isolation : pas de conflit de dépendances
- Scalabilité : lancer 10 conteneurs identiques
- CI/CD : build once, deploy anywhere
- Portabilité : du laptop au cloud sans changement

Docker dans la CI/CD

- Build : créer l'image Docker
- Push : envoyer sur un registry (Docker Hub, ECR)
- Pull : récupérer l'image en prod
- Run : lancer le conteneur
- TD2 : On conteneurise l'application Streamlit !
- TD3 : On déploira l'application sur Azure





11. Conclusion

Ce qu'on a vu

- DevOps : casser les silos Dev/Ops
- Git : versioning, branches, collaboration
- GitHub flow : fork, branch, PR, merge
- Investigation : log, blame, show
- Hotfix : corriger en prod sans embarquer le WIP
- CI/CD : automatiser build, test, deploy
- Docker : environnements reproductibles